

REPORTE QA UNIFICADO

Pruebas, Validación y Errores Documentados

Proyecto	APLIMETODOS — Métodos Numéricos
Responsables	Eduardo Leonel Monzón López — 1790-24-8668 César Gustavo Castillo García — 1790-24-20456
Entorno	Docker Compose · API .NET · Worker Python · SQL Server · Redis · Frontend React
Métodos evaluados	Raíces: Newton-Raphson, Secante, Müller Sistemas: Gauss, Gauss-Seidel, Gauss-Jordan
Tipo de pruebas	Funcionales, matemáticas, validación de entradas, visuales y de estrés/carga
Fecha	27 de mayo de 2026 (consolidado: 21, 26 y 27 de mayo)
Institución	Universidad Mariano Gálvez de Guatemala Curso 021 — Métodos Numéricos Ing. MA. Carmelo Estuardo Mayén Monterroso

1. Resumen Ejecutivo

El sistema demostró estabilidad operativa general bajo Docker. Los servicios (API, Worker, Redis, SQL Server) se mantuvieron activos frente a entradas inválidas, overflow y divisiones por cero. Los seis métodos numéricos se ejecutan y, en la mayoría de funciones bien condicionadas, convergen al valor esperado.

Las pruebas se realizaron en varias rondas. Una buena parte de los errores detectados en las primeras rondas ya fue corregida y verificada. Quedan abiertos principalmente errores de validación matemática de la convergencia y problemas visuales en las tablas de iteraciones.

Hallazgos abiertos por severidad:

Crítico	Alto	Medio	Bajo
2 abiertos	1 abierto	3 abiertos	1 abierto

2. Registro Unificado de Errores

ID	Sev.	Título	Descripción	Estado
B-01	Crítico	Falsa convergencia (sin raíz real)	Secante/Newton marcan «convergencia» en funciones con solo raíces complejas (x^2+4 , x^2+9 , x^2+25). No se verifica $ f(x^*) \approx 0$.	Abierto
B-02	Alto	Secante con $\tan(x)$ cerca de $\pi/2$	Con $\tan(x)$, $x_0=1.5$, $x_1=1.57$, reporta convergencia hacia $x=0$, ignorando la discontinuidad en $\pi/2$.	Abierto
B-03	Medio	Tabla Newton-Raphson incompleta	No se muestran de forma consistente $F(x_i)$, $F'(x_i)$ y E_a en la tabla de iteraciones.	Abierto
B-04	Medio	Tabla Müller con columnas	En raíces complejas la tabla muestra guiones en X_0 , X_1 , X_2 y $F(X_2)$ aunque	Abierto

		vacías	el backend sí calcula.	
B-05	Medio	Sistemas: celdas vacías o con texto	Gauss-Seidel/Jordan con celdas vacías genera error NoneType. Falta validar la matriz.	Abierto
B-06	Bajo	Mensaje impreciso para función inválida	Para $f(x)=\text{hola}$ se reporta «discontinuidad» en lugar de «función inválida».	Abierto
ID	Sev.	Título	Descripción	Estado
B-15	Medio	Sin límite en iteraciones/expresión	Se aceptan cantidades absurdas de iteraciones y funciones enormes con alto consumo de recursos.	Abierto
B-16	Crítico	Jobs colgados / Pending infinito	Muchos jobs rápidos, funciones oscilantes o anidadas dejan jobs permanentemente en Pending. Falta timeout.	Abierto
B-17	Crítico	Colapso parcial bajo carga	Tras pruebas extremas el frontend mostró «Network Error» y la API/Worker dejó de responder.	Abierto
B-07	Crítico	Derivada igual a cero (Newton)	Newton con x^3 , $x_0=0$ generaba NaN. Ahora detecta y avisa «derivada cercana a cero».	✓ Solucionado
B-08	Alto	Dominio matemático inválido	$\log(x)$ con $x_0=-1$ y $x^{(1/2)}$ con $x_0=-4$ terminan con «math domain error».	✓ Solucionado
B-09	Alto	División por cero en Secante ($x_0=x_1$)	Valores iniciales iguales se rechazan en el frontend y se valida el denominador.	✓ Solucionado
B-10	Medio	Validación de entradas en frontend	Texto basura, letras en campos numéricos, tolerancia negativa ahora muestran error en rojo.	✓ Solucionado
B-11	Medio	Worker fallaba al parsear el job	El worker intentaba convertir a entero el JSON de cola. Corregido con <code>json.loads()</code> .	✓ Solucionado

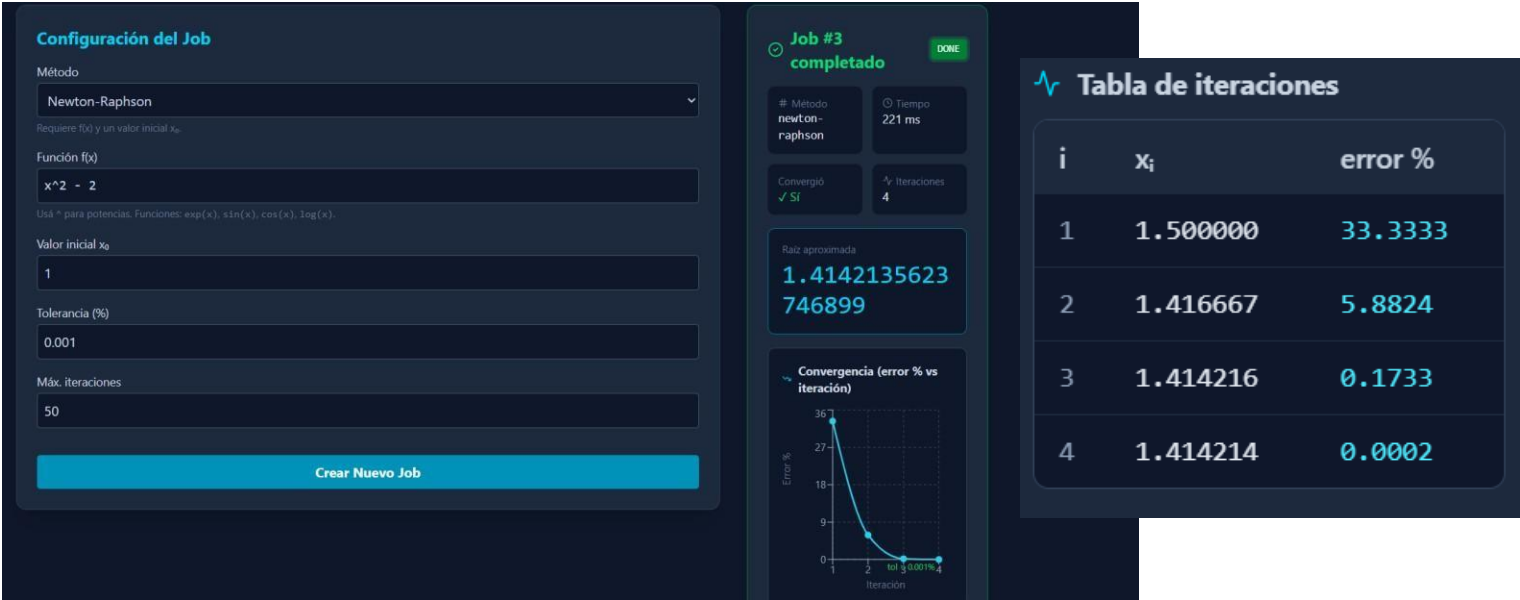
B-12	Alto	Inconsistencia iter. vs. mensaje	El sistema indicaba error en iteración 2 pero la tabla solo mostraba la iteración 1.	Pendiente verificar
B-13	Medio	Gráfica incompleta al no converger	Al no converger no se genera gráfica ni se muestran iteraciones parciales.	Pendiente verificar
B-14	Alto	Falsa convergencia con tan(x) Newton	Newton indicó convergencia cerca de $\pi/2$ con tan(x) pese a la discontinuidad.	Pendiente verificar

NEWTON RAPHSON

Prueba 1.

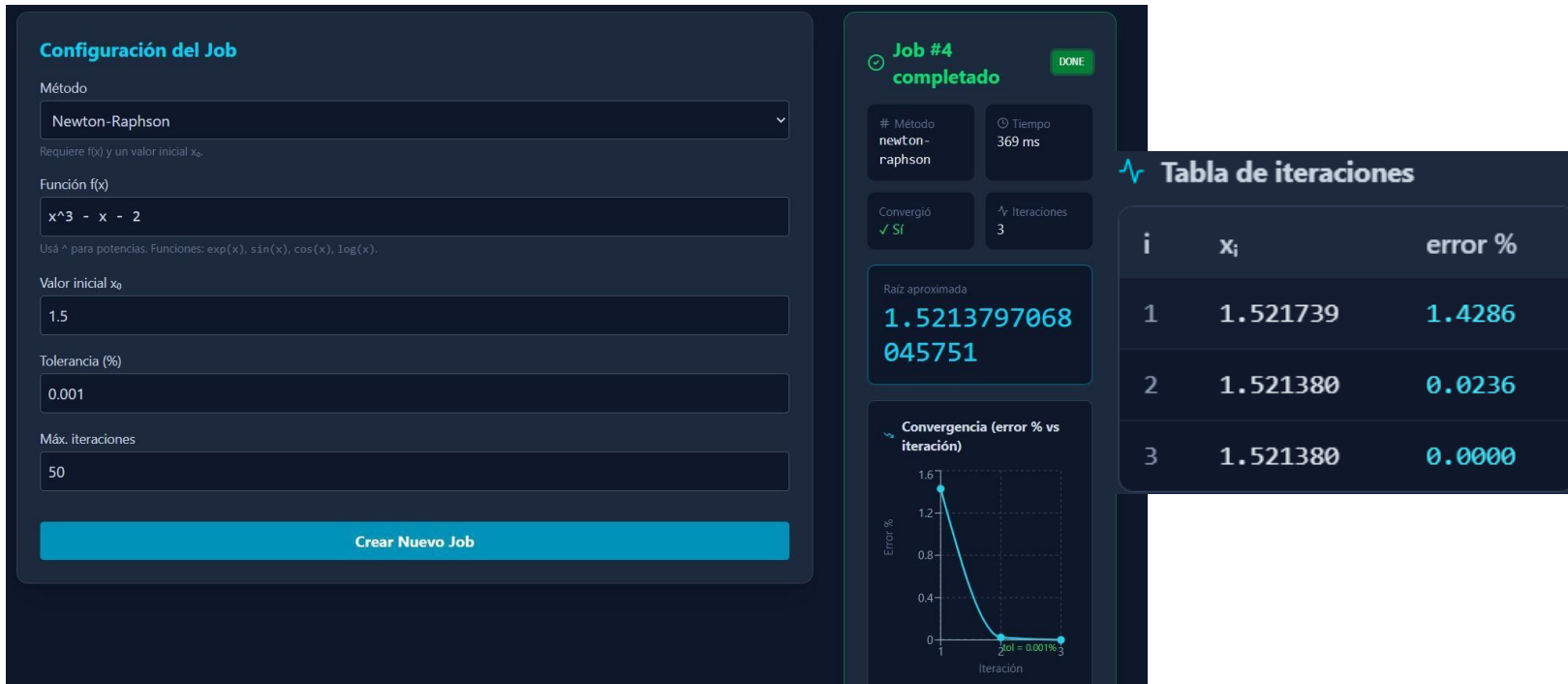
Función: $x^2 - 2$ $x_0 = 1$

Caso clásico y simple para verificar convergencia básica.

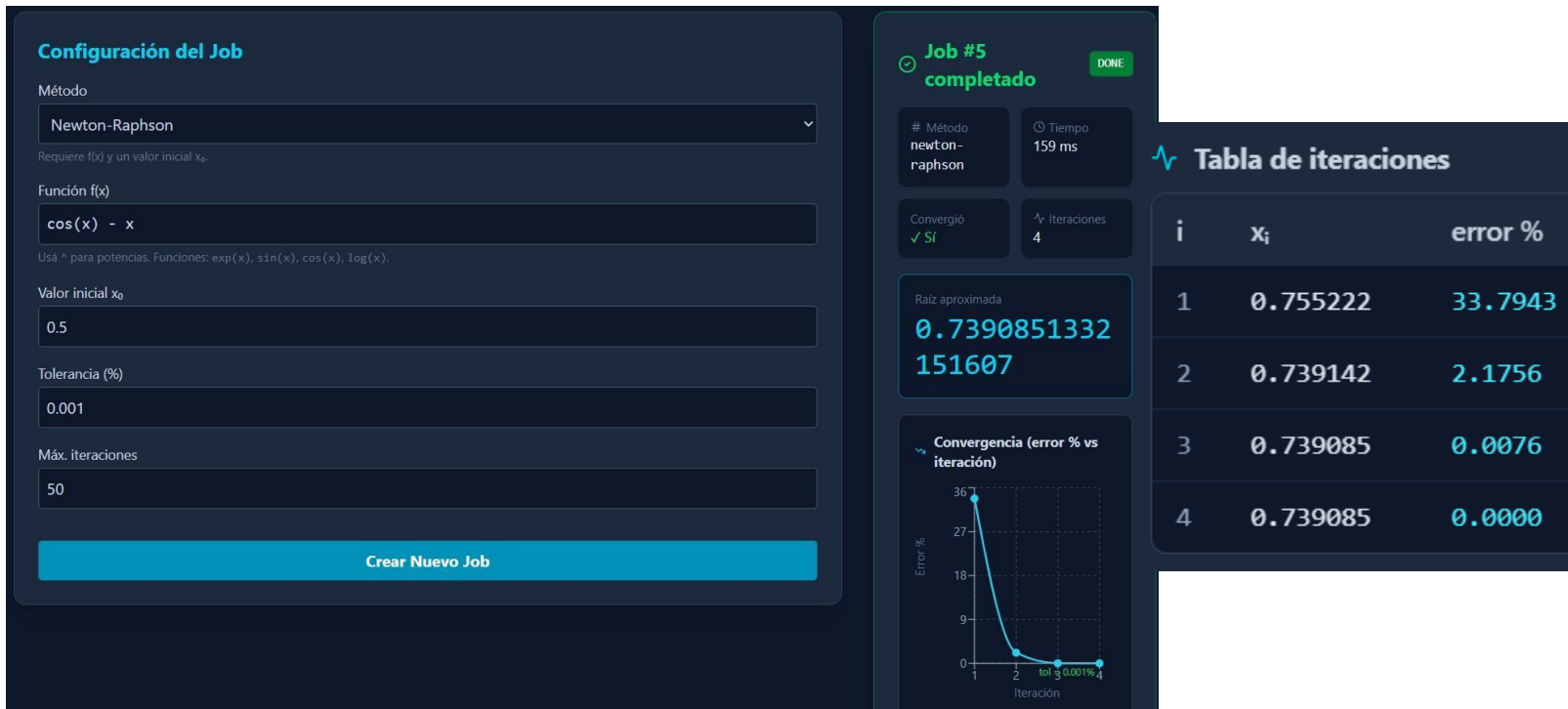


Prueba 2: función: $x^3 - x - 2$

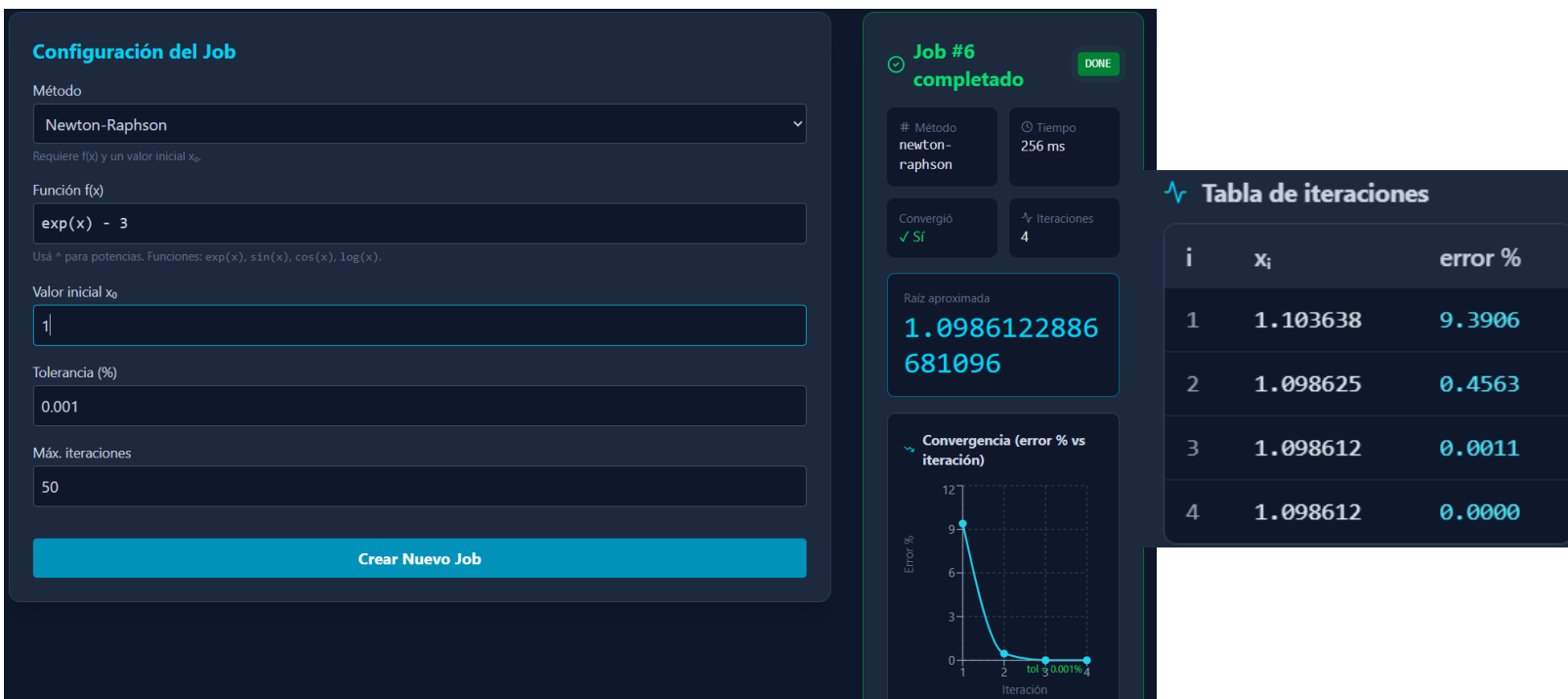
$x_0 = 1.5$ Prueba funciones polinómicas de grado mayor.



Prueba 3: función: $\cos(x) - x$ $x_0: 0.5$ Prueba de funciones trigonométricas.



Prueba 4: función: $\exp(x) - 3$ $x_0: 1$ Prueba funciones exponenciales.



Prueba 5: función: $\log(x) - 1$ x_0 : 2 prueba funciones logarítmicas

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

$\log(x) - 1$

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$.

Valor inicial x_0

2

Tolerancia (%)

0.001

Máx. iteraciones

50

Crear Nuevo Job

Job #7 completado DONE

Método

newton-raphson

⌚ Tiempo

170 ms

Convergió

✓ SI

🔢 Iteraciones

4

Raíz aproximada

2.7182818284
589376

Convergencia (error % vs iteración)

i	x_i	error %
1	2.613706	23.4803
2	2.716244	3.7750
3	2.718281	0.0749
4	2.718282	0.0000

Prueba 6:función: $x^3 - 6x^2 + 11x - 6$ x_0 : 3.5 convergencia dependiendo del punto inicial.

Configuración del Job

Método

Newton-Raphson

Requiere f(x) y un valor inicial x₀.

Función f(x)

$x^3 - 6x^2 + 11x - 6$

Usá ^ para potencias. Funciones: exp(x), sin(x), cos(x), log(x).

Valor inicial x₀

3.5

Tolerancia (%)

0.001

Máx. iteraciones

50

Crear Nuevo Job

Job #8
completado

DONE

Método
newton-raphson

⌚ Tiempo
347 ms

Convergió
✓ Sí

📈 Iteraciones
5

Raíz aproximada

3.000000000
015066

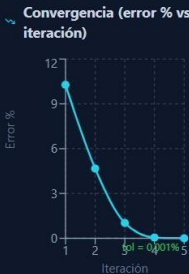


Tabla de iteraciones

i	x _i	error %
1	3.173913	10.2740
2	3.032307	4.6699
3	3.001456	1.0279
4	3.000003	0.0484
5	3.000000	0.0001

Prueba 7: función: $\sin(x)$ x_0 : 3 Prueba convergencia hacia cero.

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

$\sin(x)$

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$.

Valor inicial x_0

3

Tolerancia (%)

0.001

Máx. iteraciones

50

Crear Nuevo Job

Job #9 completado DONE

Método
newton-raphson

⌚ Tiempo
304 ms

Convergió
✓ Sí

📈 Iteraciones
3

Raíz aproximada

3.141592653589793

Convergencia (error % vs iteración)

Tabla de iteraciones

i	x_i	error %
1	3.142547	4.5360
2	3.141593	0.0304
3	3.141593	0.0000

Prueba 8: función: $x^2 - 9$ x_0 : -5 Prueba raíces negativas

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

$x^2 - 9$

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$.

Valor inicial x_0

-5

Tolerancia (%)

0.001

Máx. iteraciones

50

Crear Nuevo Job

Job #14 completado DONE

Método
newton-raphson

⌚ Tiempo
178 ms

Convergió
✓ Sí

📈 Iteraciones
5

Raíz aproximada

-3

Convergencia (error % vs iteración)

Tabla de iteraciones

i	x_i	error %
1	-3.400000	47.0588
2	-3.023529	12.4514
3	-3.000092	0.7812
4	-3.000000	0.0031
5	-3.000000	0.0000

Prueba 9: función: $x^5 - x + 1$

$x_0: -1$

Prueba estabilidad numérica.

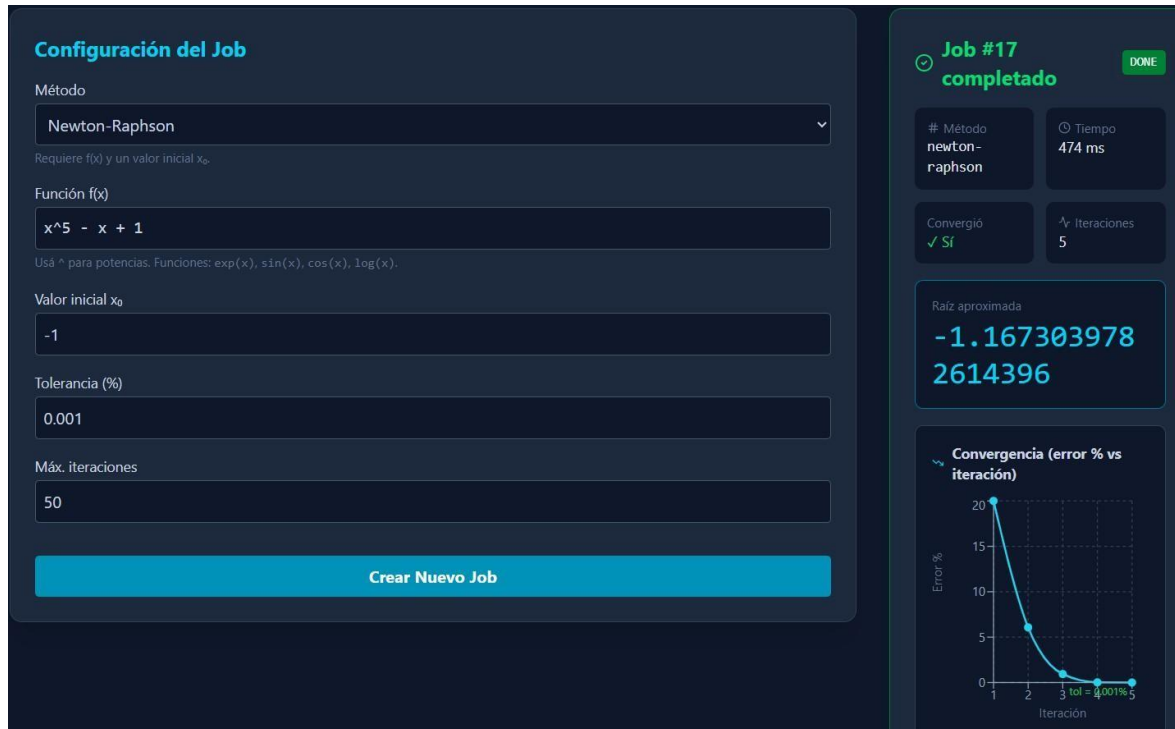


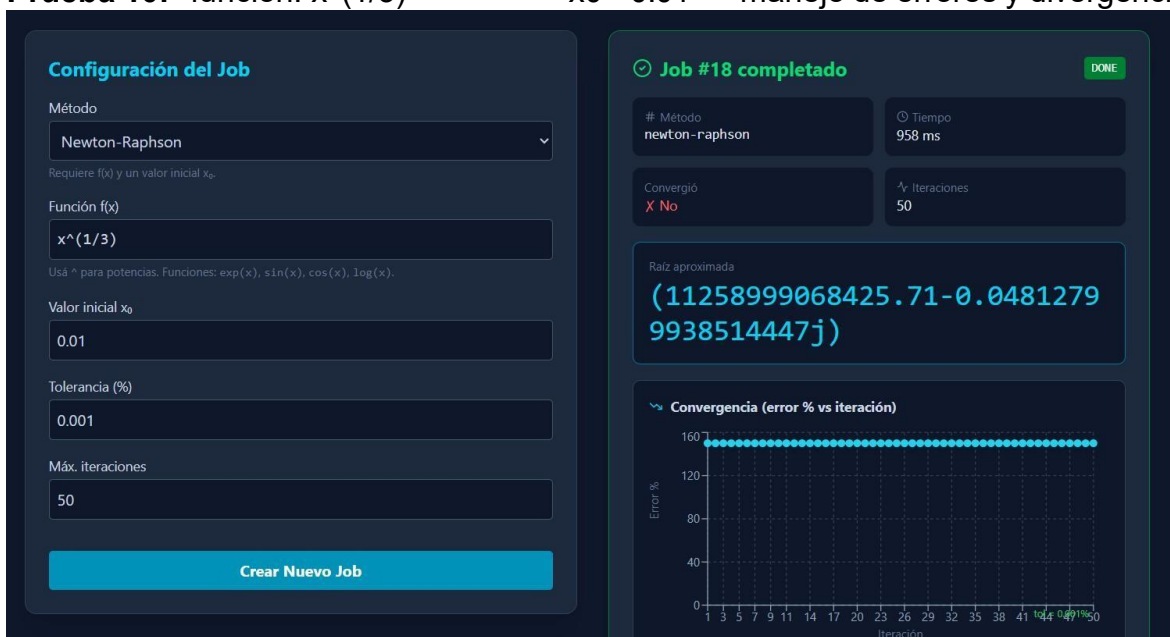
Tabla de iteraciones

i	x_i	error %
1	-1.250000	20.0000
2	-1.178459	6.0707
3	-1.167537	0.9355
4	-1.167304	0.0200
5	-1.167304	0.0000

Prueba 10: función: $x^{(1/3)}$

$x_0= 0.01$

manejo de errores y divergencia.



BUG ya que al ser una función infinita debería de mostrar error o algo parecido (a mi parecer)

SECANTE

PRUEBA 1: Funcion: $x^2 - 2$ x_0 :1 x_1 : 2 Caso básico para validar funcionamiento general.

Configuración del Job

Método

Secante

Requiere f(x) y dos valores iniciales x_0, x_1 .

Función f(x)

$x^2 - 2$

Usá ^ para potencias. Funciones: exp(x), sin(x), cos(x), log(x).

Valor inicial x_0

1

Valor inicial x_1

2

Tolerancia (%)

0.001

Máx. iteraciones

50

Crear Nuevo Job

Job #24 completado

Método
secante

⌚ Tiempo
239 ms

Convergió
✓ Sí

📈 Iteraciones
5

Raíz aproximada
1.4142135620573204

Convergencia (error % vs iteración)

Iteración	Error %
1	50.0000
2	4.7619
3	1.0345
4	0.0299
5	0.0002

i	x_i	error %
1	1.333333	50.0000
2	1.400000	4.7619
3	1.414634	1.0345
4	1.414211	0.0299
5	1.414214	0.0002

PRUEBA 2: Funcion: $x^3 - x - 2$ x_0 : 1 x_1 : 2 Prueba convergencia normal.

Configuración del Job

Método

Secante

Requiere f(x) y dos valores iniciales x_0, x_1 .

Función f(x)

$x^3 - x - 2$

Usá ^ para potencias. Funciones: exp(x), sin(x), cos(x), log(x).

Valor inicial x_0

1

Valor inicial x_1

2

Tolerancia (%)

0.001

Máx. iteraciones

50

Crear Nuevo Job

Job #25 completado

Método
secante

⌚ Tiempo
366 ms

Convergió
✓ Sí

📈 Iteraciones
6

Raíz aproximada
1.5213797079848717

Convergencia (error % vs iteración)

Iteración	Error %
1	50.0000
2	8.8435
3	4.4726
4	0.6735
5	0.0296
6	0.0002

i	x_i	error %
1	1.333333	50.0000
2	1.462687	8.8435
3	1.531169	4.4726
4	1.520926	0.6735
5	1.521376	0.0296
6	1.521380	0.0002

Página 11

PRUEBA 3: Funcion: $\cos(x) - x$ $x_0: 0$ $x_1: 1$ Prueba funciones trigonométricas.

Configuración del Job

Método

Secante

Requiere f(x) y dos valores iniciales x_0, x_1 .

Función f(x)

$\cos(x) - x$

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$.

Valor inicial x_0

0

Valor inicial x_1

1

Tolerancia (%)

0.001

Máx. iteraciones

50

Crear Nuevo Job

Job #26 completado

DONE

Método: secante

⌚ Tiempo: 206 ms

Convergió:

✓ Sí

📈 Iteraciones: 5

Raíz aproximada

0.7390851332150012

Convergencia (error % vs iteración)



Tabla de iteraciones

i	x_i	error %
1	0.685073	45.9698
2	0.736299	6.9572
3	0.739119	0.3816
4	0.739085	0.0046
5	0.739085	0.0000

PRUEBA 4: Funcion: $\exp(x) - 2$ $x_0: 0$ $x_1: 1$ Prueba estabilidad con exponenciales.

Configuración del Job

Método

Secante

Requiere f(x) y dos valores iniciales x_0, x_1 .

Función f(x)

$\exp(x) - 2$

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$.

Valor inicial x_0

0

Valor inicial x_1

1

Tolerancia (%)

0.001

Máx. iteraciones

50

Crear Nuevo Job

Job #27 completado

DONE

Método: secante

⌚ Tiempo: 129 ms

Convergió:

✓ Sí

📈 Iteraciones: 6

Raíz aproximada

0.6931471805599592

Convergencia (error % vs iteración)



Tabla de iteraciones

i	x_i	error %
1	0.581977	71.8282
2	0.676693	13.9969
3	0.694081	2.5053
4	0.693139	0.1359
5	0.693147	0.0011
6	0.693147	0.0000

PRUEBA 5:

Funcion: x^3 x_0 : 0 x_1 : 0.0001

detectar divisiones entre cero o divergencia

Configuración del Job

Método

Secante

Requiere $f(x)$ y dos valores iniciales x_0, x_1 .

Función $f(x)$

x^3

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$.

Valor inicial x_0

0

Valor inicial x_1

0.0001

Tolerancia (%)

0.001

Máx. iteraciones

50

Crear Nuevo Job

Job #28

completado

DONE

Método

secante

⌚ Tiempo

300 ms

Convergió

✓ Sí

📈 Iteraciones

2

Raíz aproximada

0

Convergencia (error % vs iteración)

Tabla de iteraciones

i	x_i	error %
1	0.000000	0.0100
2	0.000000	0.0000

ERRORES ENCONTRADOS:

Configuración del Job

Método
Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$
 x^3

Usá $^$ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$.

Valor inicial x_0
0

Tolerancia (%)
0.001

Máx. iteraciones
50

Crear Nuevo Job

Job #29 completado

Método: newton-raphson

⌚ Tiempo: 405 ms

Convergió: **X No**

↕ Iteraciones: —

Raíz aproximada

Derivada cercana a cero en iteración 1 ($f'(0.0) = 0.0$). División por cero.

DERIVADA IGUAL A 0: En esta prueba se utilizó la función x^3 con valor inicial $x_0 = 0$ para verificar el comportamiento del método de Newton-Raphson cuando la derivada de la función es igual a cero. Debido a que el método utiliza una división entre la derivada evaluada en el punto actual, el sistema puede generar una división entre cero, produciendo errores como NaN, infinito o una excepción matemática.

Configuración del Job

Método
Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$
 $\log(x)$

Usá $^$ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$.

Valor inicial x_0
-1

Tolerancia (%)
0.001

Máx. iteraciones
50

Crear Nuevo Job

Job #30 completado

Método: newton-raphson

⌚ Tiempo: 186 ms

Convergió: **X No**

↕ Iteraciones: —

Raíz aproximada

Error de evaluación en iteración 1: math domain error

Se probó la función $\log(x)$ utilizando un valor inicial negativo ($x_0 = -1$) para comprobar cómo el sistema maneja funciones fuera de su dominio matemático. Como el logaritmo natural únicamente

está definido para números mayores que cero, el cálculo puede generar errores de dominio, valores indefinidos o fallos en el procesamiento del worker

Configuración del Job

Método
Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$
 $x^{(1/2)}$

Usa ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$.

Valor inicial x_0
-4

Tolerancia (%)
0.001

Máx. iteraciones
50

Crear Nuevo Job

Job #31 completado DONE

Método: newton-raphson

⌚ Tiempo: 320 ms

Convergió: X No

⌚ Iteraciones: —

Raíz aproximada: —

Error de evaluación en iteración 1: math domain error

En esta prueba se utilizó la función $x^{(1/2)}$ con un valor inicial negativo para provocar resultados complejos. El objetivo fue comprobar si el sistema soporta números complejos o si presenta fallos en la serialización, almacenamiento en base de datos o representación gráfica

Configuración del Job

Método

Secante

Requiere $f(x)$ y dos valores iniciales x_0, x_1 .

Función $f(x)$

$\exp(x) - 2$

Usá $^$ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$.

Valor inicial x_0

2

Valor inicial x_1

2

Tolerancia (%)

0.001

Máx. iteraciones

50

Crear Nuevo Job

Job #32 completado

DONE

Método

secante

⌚ Tiempo

76 ms

Convergió

X No

⌚ Iteraciones

—

Raíz aproximada

—

Denominador cercano a cero en iteración 1 ($f(x_i) - f(x_{i-1}) = 0.0$). División por cero.

En esta prueba del método de la secante se utilizaron valores iniciales idénticos ($x_0 = x_1$) para provocar una división entre cero dentro de la fórmula iterativa. El objetivo fue validar si el sistema detecta correctamente configuraciones inválidas antes de iniciar el cálculo y si maneja adecuadamente el error generado.

PRUEBA (2) DE PROGRAMA PARA POSIBLES ERRORES

Encargados de pruebas:

Eduardo Leonel Monzón López 1790-24-
8668 César Gustavo Castillo García
1790-24-20456

PRUEBAS EN VISUAL STUDIO CODE (PYTHON)

Primera prueba Error estimado:

Error en la casillaa donde se ingresa la función, esta deja ingresar valores no admitidos y genera un Job.

The screenshot displays the 'Busqueda de Raíces' (Root Finding) web application interface. The left panel, titled 'Configuración del Job', contains the following fields:

- Método:** A dropdown menu set to 'Newton-Raphson'.
- Requiere f(x) y un valor inicial x_0 .**
- Función f(x):** A text input field containing a comma (`,`).
- Usá * para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$.**
- Valor inicial x_0 :** A text input field containing the value `1`.
- Tolerancia (%):** A text input field containing the value `0.001`.
- Máx. iteraciones:** A text input field containing the value `50`.
- Crear Nuevo Job:** A large blue button at the bottom of the configuration panel.

The right panel, titled 'Job #25 completado' with a green 'DONE' button, shows the results of the job:

- # Método:** newton-raphson
- ⌚ Tiempo:** 4 ms
- Convergió:** X No
- ⬆ Iteraciones:** —
- Raíz aproximada:** —
- Error message:** Error al parsear la expresión: Simplify of expression 'could not parse ',' failed, because of exception being raised: SyntaxError: invalid syntax (<string>, line 1)

segunda prueba

Error estimado: presenta error en las casillas, en este caso en funcion, valor inicial1 y valor inicial2. Permitiendo el ingreso de esos valores y generando un Job inservible

(Consideracion: no permitir que genere Jobs con valores no adecuados)

Configuración del Job

Método
Müller

Requiere $f(x)$ y tres valores iniciales x_0, x_1, x_2 . Soporta raíces complejas.

Función $f(x)$
,

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$.

Valor inicial x_0
1

Valor inicial x_1
hola

Valor inicial x_2
jeje

Tolerancia (%)
0.001

Máx. iteraciones
50

Crear Nuevo Job

Job #26 completado

Método
muller

⌚ Tiempo
2 ms

Convergió
X No

📈 Iteraciones
—

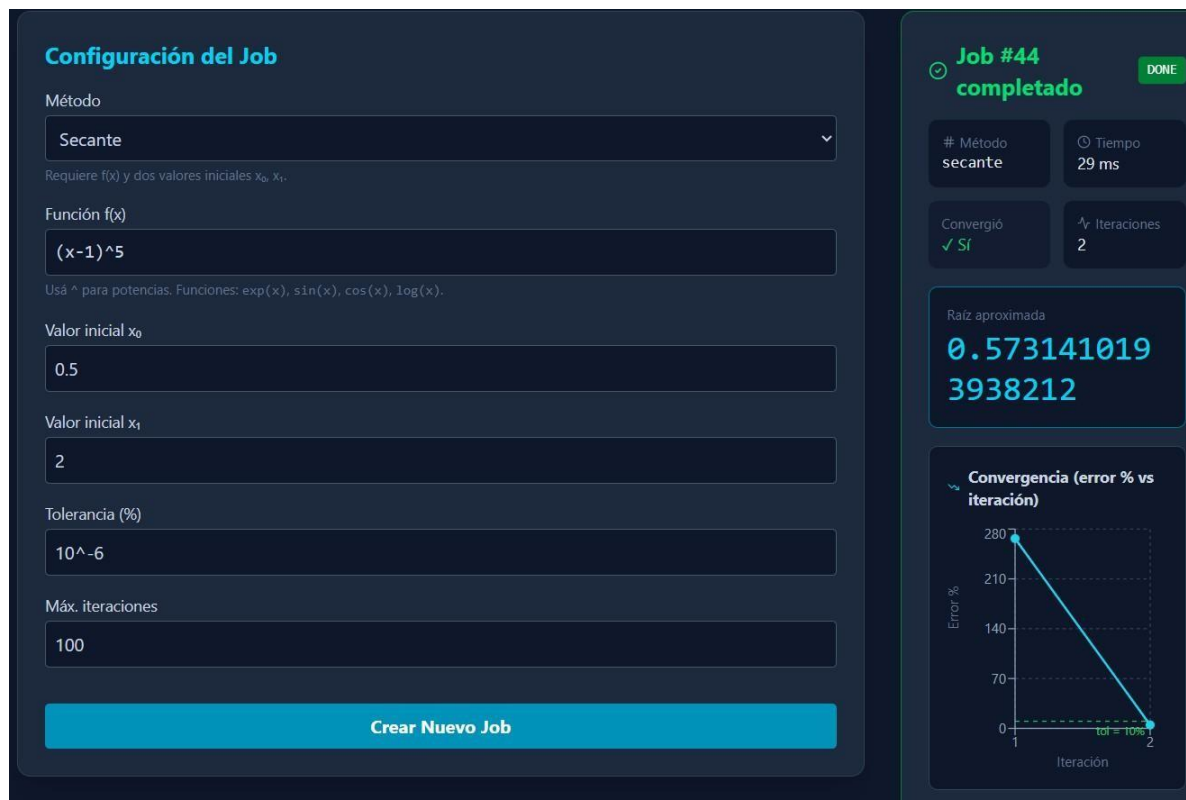
Raíz aproximada
—

Error al parsear la expresión: Sympify of expression 'could not parse :"' failed, because of exception being raised: SyntaxError: invalid syntax (<string>, line 1)

Tercera prueba Error

estimado:

El método de la secante aplicado a $f(x)=(x-1)^5$ debería converger lentamente hacia $x=1$. Sin embargo, el programa reporta convergencia en solo dos iteraciones con una raíz incorrecta ($x \approx 0.5731$). Esto sugiere un error en la implementación del cálculo de la función o del criterio de convergencia.



Posible causa del bug:

- El programa podría estar **evaluando mal la función** (por ejemplo, usando ^ como operador de bitwise XOR en lugar de potencia).
- O podría estar **comparando el error relativo incorrectamente**, deteniéndose antes de tiempo.
- También puede haber un **problema en la actualización de los valores** x_0, x_1 o en el cálculo de la pendiente secante.

Cuarta prueba Error estimado:

El método de Müller aplicado a $f(x)=e-x-x$ debería converger en aproximadamente 6 iteraciones hacia la raíz $x \approx 0.5671433$. Sin embargo, el programa reporta que no converge y no muestra resultados. Esto indica un error en la implementación del método, ya que la función es continua y tiene una raíz real bien definida. El fallo puede deberse a un manejo incorrecto de la función exponencial, del criterio de parada o de la actualización de los puntos iniciales.

Configuración del Job

Método

Müller

Requiere $f(x)$ y tres valores iniciales x_0, x_1, x_2 . Soporta raíces complejas.

Función $f(x)$

$e^x - x - x$

Usá $^$ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$.

Valor inicial x_0

1

Valor inicial x_1

1.5

Valor inicial x_2

2

Tolerancia (%)

10^{-6}

Máx. iteraciones

50

Job #45 completado

DONE

# Método	⌚ Tiempo
müller	79 ms
Convergió	🔄 Iteraciones
X No	—
Raíz aproximada	
—	

Error de evaluación en iteración 1: Cannot convert expression to float

OBSERVACION GENERAL

Método de la Secante con funciones de raíz múltiple

- Ejemplo: $f(x)=(x-1)^5$ con valores iniciales $x_0=0.5$, $x_1=2$.
 - **Resultado esperado:** convergencia lenta hacia $x \approx 1$ en más de 30 iteraciones.
 - **Resultado obtenido:** el programa reportó convergencia en solo 2 iteraciones hacia $x \approx 0.5731$, lo cual es incorrecto.
-
- **Conclusión:** existe un error en el cálculo de la función o en el criterio de parada, ya que la raíz real es $x=1$.

Método de Müller con funciones continuas y raíz real conocida

- Ejemplo: $f(x)=e^{-x}-x$ con valores iniciales $x_0=0$, $x_1=0.5$, $x_2=1$.
 - **Resultado esperado:** convergencia estable hacia $x \approx 0.5671433$ en 5–7 iteraciones.
 - **Resultado obtenido:** el programa reportó “no converge” y no mostró ni raíz, ni iteraciones, ni gráfica.
-
- **Conclusión:** el método debería converger, por lo que el fallo indica un error en la implementación del algoritmo de Müller (posible problema en la actualización de los puntos o en el manejo de la función exponencial).

Comportamiento del programa cuando no hay convergencia

- Se comprobó que, en los casos donde el programa determina que no converge, **no se genera gráfica ni se muestran iteraciones**.
 - Esto dificulta el análisis, ya que no permite visualizar el comportamiento del método antes de fallar.
-
- **Conclusión:** se recomienda que el programa muestre al menos las iteraciones realizadas y la gráfica parcial, incluso si no se alcanza la convergencia, para facilitar la detección de errores.

El programa **sí ejecuta** los métodos, pero presenta inconsistencias en casos de raíces múltiples y funciones continuas con soluciones conocidas

Se recomienda revisar:

- El cálculo de la función (especialmente potencias y exponenciales).
- El criterio de parada y manejo de tolerancia.
- La actualización de los puntos en el método de Müller.
- La generación de gráficas y reportes de iteraciones, incluso en casos de divergencia.

Y revisar las casillas donde el programa devuelve un Job a pesar que no son valores admitidos-

PRUEBA DEL SISTEMA PARA POSIBLES ERRORES

ENCARGADOS:

Eduardo Leonel Monzón López 1790-24-

8668 César Gustavo Castillo García

1790-24-20456

PRUEBAS NEWTON RAPHSON

Pruebas de funciones, comportamiento lógico:

Caso 1: convergencia básica - función: $x^2 - 2$

x_0 : 1

NINGUN ERROR O BUG ENCONTRADO

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

$x^2 - 2$

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

1

Tolerancia (%)

0.001

Acepta 0.001, $1e-6$ o 10^{-6} .

Máx. iteraciones

50

Crear Nuevo Job

Job #3004 completado

Método newton-raphson

⌚ Tiempo 903 ms

Convergió ✓ Sí

📈 Iteraciones 4

Raíz aproximada

1.4142135623746899

Convergencia (error % vs iteración)

Tabla de iteraciones		
i	x_i	error %
1	1.500000	33.3333
2	1.416667	5.8824
3	1.414216	0.1733
4	1.414214	0.0002

Caso 2: funcion trigonométrica: funcion: $\cos(x)-x$

x_0 : 1 NO SE ENCONTRO ERROR O BUG

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

$\cos(x) - x$

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

1

Tolerancia (%)

0.001

Acepta 0.001, $1e-6$ o 10^{-6} .

Máx. iteraciones

50

Crear Nuevo Job

Job #3005 completado

Método newton-raphson

⌚ Tiempo 791 ms

Convergió ✓ Sí

📈 Iteraciones 4

Raíz aproximada

0.7390851332151607

Convergencia (error % vs iteración)

Tabla de iteraciones		
i	x_i	error %
1	0.750364	33.2687
2	0.739113	1.5222
3	0.739085	0.0038
4	0.739085	0.0000

Caso 3: exponencial – funcion: $e^x - 3$

Xo: 1 NO SE ENCONTRO ERROR O BUG

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

$e^x - 3$

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

1

Tolerancia (%)

0.001

Acepta 0.001, 1e-6 o 10^-6.

Máx. iteraciones

50

Crear Nuevo Job

Job #3006 completado DONE

Método: newton-raphson

⌚ Tiempo: 797 ms

Convergió: **✓ Sí**

📈 Iteraciones: 4

Raíz aproximada

1.0986122886681096

Convergencia (error % vs iteración)

Tabla de iteraciones

i	x_i	error %
1	1.103638	9.3906
2	1.098625	0.4563
3	1.098612	0.0011
4	1.098612	0.0000

Convergencia alcanzada en 4 iteraciones.

PRUEBAS SECANTE

Caso 1: secante normal

funcion: $x^3 - x - 2$ Xo: 1 X1: 2 NO SE ENCONTRO ERROR O BUG

Configuración del Job

Método

Secante

Requiere $f(x)$ y dos valores iniciales x_0 , x_1 .

Función $f(x)$

$x^3 - x - 2$

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

1

Valor inicial x_1

2

Tolerancia (%)

0.001

Acepta 0.001, 1e-6 o 10^-6.

Máx. iteraciones

50

Crear Nuevo Job

Job #3007 completado DONE

Método: secante

⌚ Tiempo: 471 ms

Convergió: **✓ Sí**

📈 Iteraciones: 6

Raíz aproximada

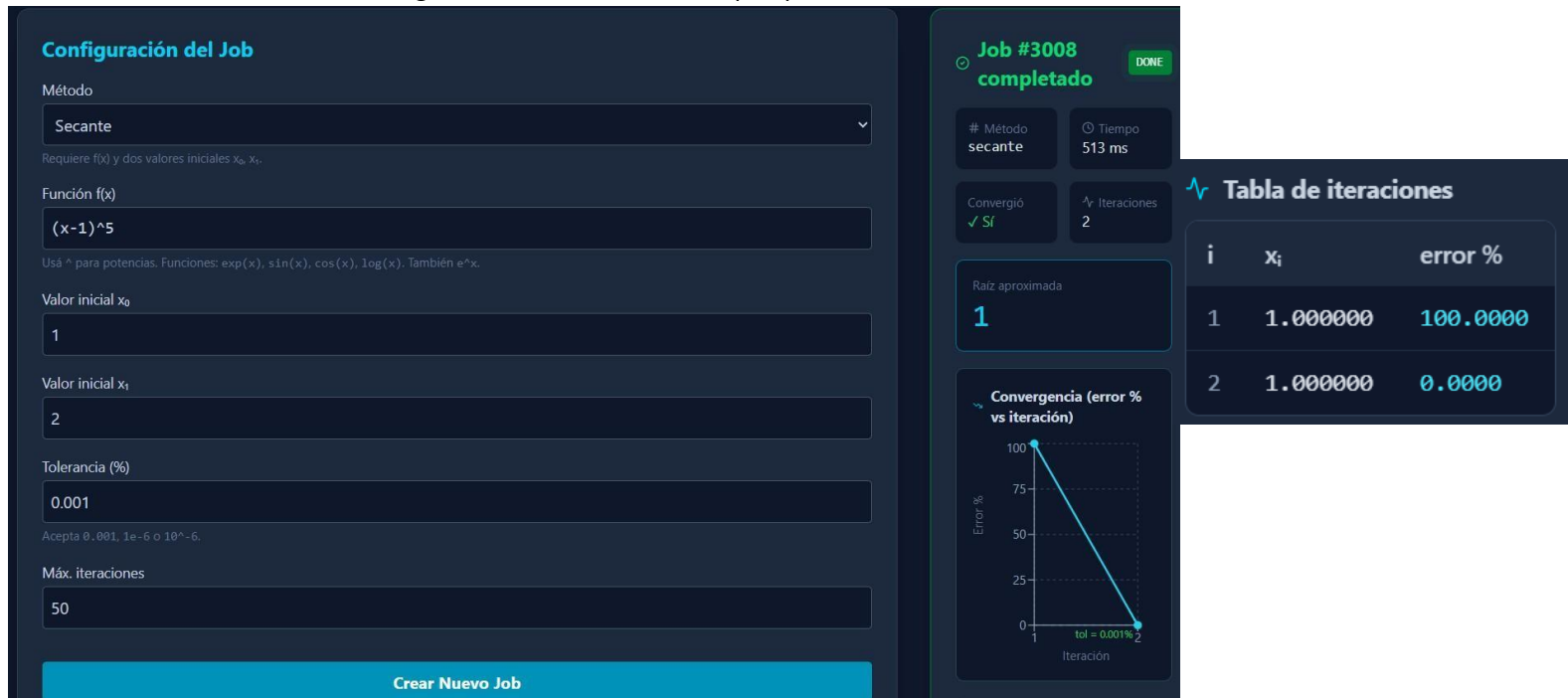
1.5213797079848717

Convergencia (error % vs iteración)

Tabla de iteraciones

i	x_i	error %
1	1.333333	50.0000
2	1.462687	8.8435
3	1.531169	4.4726
4	1.520926	0.6735
5	1.521376	0.0296
6	1.521380	0.0002

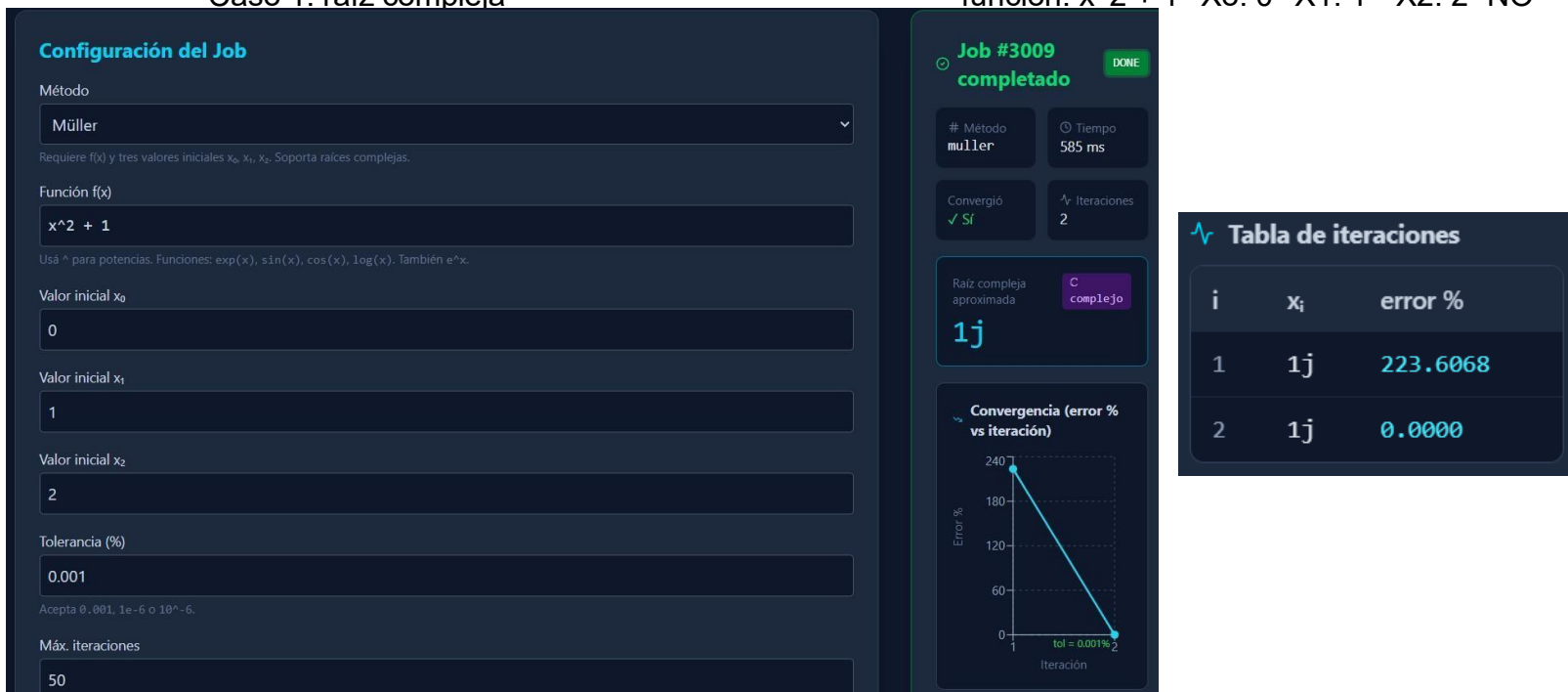
Caso 2: convergencia lenta función: $(x-1)^5$ $X_0: 1$ $X_1: 2$ NO SE ENCONTRO ERROR O BUG



PRUEBAS MULLER

Caso 1: raíz compleja

funcion: $x^2 + 1$ $X_0: 0$ $X_1: 1$ $X_2: 2$ NO



Caso 2: exponencial funcion: $e^{-x} - x$ X0: 0 X1:1 X2: 2 NO SE ENCONTRO ERROR O BUG

Configuración del Job

Método

Müller

Requiere $f(x)$ y tres valores iniciales x_0, x_1, x_2 . Soporta raíces complejas.

Función $f(x)$

$e^{-x} - x$

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

0

Valor inicial x_1

1

Valor inicial x_2

2

Tolerancia (%)

0.001

Acepta 0.001, 1e-6 o 10^-6.

Máx. iteraciones

50

Job #3010 completado

DONE

Método

müller

⌚ Tiempo

463 ms

Convergió

✓ Sí

↕ Iteraciones

4

Raíz aproximada

0.567143290409927

Convergencia (error % vs iteración)

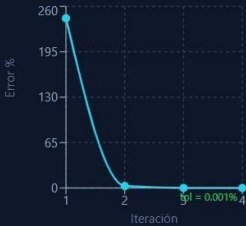


Tabla de iteraciones		
i	x_i	error %
1	0.5829393389950392	243.0889
2	0.5667522013339478	2.8561
3	0.5671436758354566	0.0690
4	0.567143290409927	0.0001

PRUEBAS GENERALES

Derivada 0: SOLUCIONADO

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

x^3

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

0

Tolerancia (%)

0.001

Acepta 0.001, 1e-6 o 10^-6.

Máx. iteraciones

50

Job #3011 completado

DONE

⚠ El método no convergió.

El método se detuvo antes de generar iteraciones (ver mensaje abajo).

Método

newton-raphson

⌚ Tiempo

88 ms

Convergió

✗ No

↕ Iteraciones

—

Derivada cercana a cero en iteración 1 ($f'(0,0) = 0,0$).
División por cero.

APLIMETODOS — Reporte QA Unificado · UMG Curso 021

Página 27

Dominio invalido: SOLUCIONADO

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

$\log(x)$

Usá $^$ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

-1

Tolerancia (%)

0.001

Acepta 0.001, $1e-6$ o 10^{-6} .

Máx. iteraciones

50

Crear Nuevo Job

Job #3012 completado

El método no convergió.
El método se detuvo antes de generar iteraciones (ver mensaje abajo).

# Método	newton-raphson	⌚ Tiempo	160 ms
Convergió	X No	↕ Iteraciones	—

Error de evaluacion en iteracion 1: math domain error

División entre 0 en secante: SOLUCIONADO

Configuración del Job

Método

Secante

Requiere $f(x)$ y dos valores iniciales x_0 , x_1 .

Función $f(x)$

$\log(x)$

Usá $^$ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

2

Valor inicial x_1

2

x_1 debe ser distinto de x_0 .

Tolerancia (%)

0.001

Acepta 0.001, $1e-6$ o 10^{-6} .

Máx. iteraciones

50

Crear Nuevo Job

Texto basura: SOLUCIONADO

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

JEJEJEJE

La función debe depender de x (ej. $x^2 - 4$).

Valor inicial x_0

2

Tolerancia (%)

0.001

Acepta 0.001, $1e-6$ o 10^{-6} .

Máx. iteraciones

50

Crear Nuevo Job

Letras en números: SOLUCIONADO

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

JEJEJEJE

La función debe depender de x (ej. $x^2 - 4$).

Valor inicial x_0

EEEEEEEE

Debe ser un número válido (ej. 1.5).

Tolerancia (%)

0.001

Acepta 0.001, $1e-6$ o 10^{-6} .

Máx. iteraciones

50

Crear Nuevo Job

Tolerancia negativa: SOLUCIONADO

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

x^3

Usá $^$ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

1

Tolerancia (%)

-0.001

La tolerancia debe ser mayor que 0.

Máx. iteraciones

50

Crear Nuevo Job

Dominio invalido: CORRECTO

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

$\log(x)$

Usá $^$ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

-1

Tolerancia (%)

0.001

Acepta 0.001, $1e-6$ o 10^{-6} .

Máx. iteraciones

50

Crear Nuevo Job

Job #3013 DONE

completado

El método no convergió.
El método se detuvo antes de generar iteraciones (ver mensaje abajo).

# Método	⌚ Tiempo
newton-raphson	323 ms
Convergió	↕ Iteraciones
X No	—

Error de evaluación en iteración 1: math domain error

Interacciones negativas: SOLUCIONADO

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

$\log(x)$

Usá $^$ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

1

Tolerancia (%)

0.001

Acepta 0.001, $1e-6$ o 10^{-6} .

Máx. iteraciones

-50

Debe ser un entero mayor o igual a 1.

Crear Nuevo Job

REPORTE DE PRUEBAS Y ERRORES/BUGS

Proyecto: Métodos Numéricos

Responsable de pruebas:

Eduardo Leonel Monzón López 1790-

24-8668 Cesar Gustavo Castillo

Garcia 1790-24-20456 **Fecha:**

21/05/2026

RESUMEN

Durante las pruebas de estrés y validación realizadas sobre los métodos Newton-Raphson, Secante y Müller, se detectaron múltiples comportamientos incorrectos relacionados con convergencia falsa, bloqueo de jobs, ausencia de validaciones, fallos del parser y colapso parcial del sistema bajo carga extrema.

BUGS ENCONTRADOS:

1. Inconsistencia entre iteraciones y mensaje

El sistema indicó error en iteración 2, pero la tabla solo mostraba iteración 1.

2. Gráfica incompleta en fallos

La gráfica no representó correctamente las iteraciones cuando el método falló.

3. Falsa convergencia con $\tan(x)$

Newton-Raphson indicó convergencia cerca de $\pi/2$, aunque $\tan(x)$ tiene discontinuidad vertical.

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

$\tan(x)$

Usá $^$ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

1.570796

Tolerancia (%)

0.001

Acepta 0.001, $1e-6$ o 10^{-6} .

Máx. iteraciones

50

Crear Nuevo Job

Job #8 completado

Método
newton-raphson

Convergió
✓ Sí

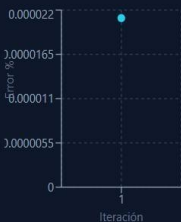
✓ Raíz aproximada
1.5707956732051036

Convergencia (error % vs iteración)

Tabla de iteraciones

⌚ Tiempo
25 ms

🔢 Iteraciones
1



4. Falta de validación en iteraciones máximas

El sistema aceptó cantidades absurdamente grandes de iteraciones y funciones lo cual genero un consumo grande, provocando problemas en el Sistema (recomendando colocar un limite o una advertencia)

Configuración del Job

Método
Newton-Raphson
Requiere f(x) y un valor inicial x₀.

Función f(x)
xx
Usá ^ para potencias. Funciones: exp(x), sin(x), cos(x), log(x). También e^x.

Valor inicial x₀
1

Tolerancia (%)
0.001
Acepta 0.001, 1e-6 o 10^-6.

Máx. iteraciones
50

Crear Nuevo Job

Procesando Job #12

Estado: Pending

Polling cada 5 segundos...

[illegible]

5. Jobs colgados por spam

Al generar múltiples jobs rápidamente, el programa se congelo y todos quedaron permanentemente en estado Pending.

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

xx

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

1

Tolerancia (%)

0.001

Acepta 0.001, $1e-6$ o 10^{-6} .

Máx. iteraciones

50

Crear Nuevo Job



Procesando Job #103

Estado: Pending

Polling cada 5 segundos...

6. Parser colgado por expresiones anidadas

La expresión (((((((((x)))))))) dejó el job en estado Pending.

Configuración del Job

Método

Newton-Raphson

Requiere $f(x)$ y un valor inicial x_0 .

Función $f(x)$

((((((((((x))))))))))

Usá ^ para potencias. Funciones: $\exp(x)$, $\sin(x)$, $\cos(x)$, $\log(x)$. También e^x .

Valor inicial x_0

1

Tolerancia (%)

0.001

Acepta 0.001, $1e-6$ o 10^{-6} .

Máx. iteraciones

50

Crear Nuevo Job



Procesando Job #104

Estado: Pending

Polling cada 5 segundos...

7. Ausencia de timeout

Los jobs largos no fueron cancelados automáticamente.

8. Pending infinito con $f(x)=x^{1/3}$

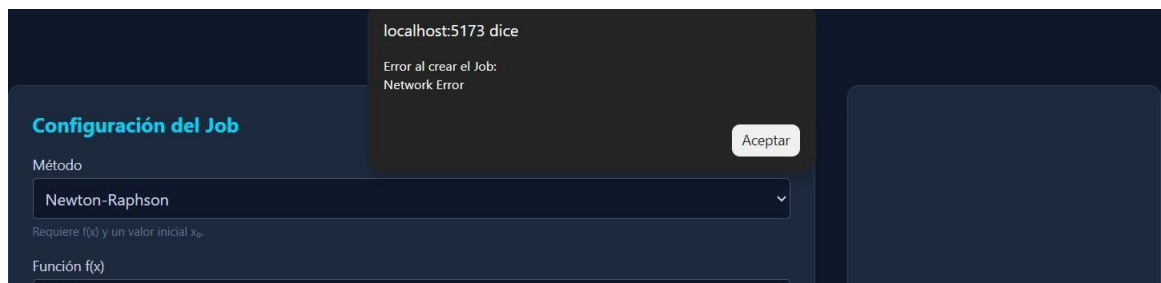
Newton-Raphson quedó permanentemente en Pending con comportamiento oscilante.

9. Pending con Müller y $\tan(x)$

El método Müller quedó colgado al trabajar con funciones discontinuas.

10. Colapso parcial del sistema

Después de múltiples pruebas extremas, el sistema presentó Network Error y dejó de responder correctamente.



```
request returned 500 Internal Server Error for API route and version http://%2F%2F.%2Fpipe%2FdockerDesktopLinuxEngine/v1.54/containers/json?all=1&filters=%7B%22label%22%3A%7B%22com.docker.compose.config-hash%22%3Atrue%2C%22com.docker.compose.oneoff%22%3Afalse%22%3Atrue%2C%22com.docker.compose.project%22%3Aapl%22%3Atrue%2C%22com.docker.compose.service%22%3Amain%22%3Atrue%7D%7D, check if the server supports the requested API version
```

Comportamiento actual (Resultado): El frontend muestra "Error de conexión" y el contenedor de la API/Worker deja de responder o se reinicia.

Comportamiento esperado: El sistema debería mostrar un mensaje de "Procesando..." o un aviso de "La función es demasiado compleja para calcular" en lugar de perder la conexión.

Conclusiones

Las pruebas realizadas permitieron identificar debilidades importantes en el manejo de convergencia, validación de entradas, control de recursos y estabilidad general del sistema. Se recomienda implementar límites de iteraciones, timeouts, mejor manejo de errores y validaciones adicionales para evitar bloqueos y colapsos del backend.

Plan de Pruebas QA — APLIMETODOS

Plataforma de Ejecución, Trazabilidad y Comparación de Métodos
Numéricos Curso 021 — Métodos Numéricos · Universidad Mariano
Gálvez

Batería de pruebas para validar el sistema según los requisitos del enunciado oficial (rúbrica de calificación y manejo de casos de error de la sección 5.2). Cada caso indica los valores a ingresar y el resultado esperado.

1. Métodos de raíces — deben CONVERGER correctamente

Método	Función	Valores	Resultado esperado
Newton-Raphson	$x^2 - 2$	$x_0=1$	$\sqrt{2} \approx 1.4142$
Newton-Raphson	$\cos(x) - x$	$x_0=0.5$	≈ 0.739
Newton-Raphson	$e^x - 3$	$x_0=1$	$\ln(3) \approx 1.0986$
Newton-Raphson	$e^{-x} - x$	$x_0=0$	≈ 0.567 (verifica fix de e^x)
Secante	$x^3 - x - 2$	$x_0=1, x_1=2$	≈ 1.5214
Secante	$(x-1)^5$	$x_0=0.5, x_1=2, \text{tol}=1e-6$	debe acercarse a $x=1$ (verifica fix de tolerancia)
Müller	$x^2 + 1$	$x_0=0, x_1=1, x_2=2$	raíz compleja $\approx i$
Müller	$e^{-x} - x$	$x_0=1, x_1=1.5, x_2=2$	≈ 0.567

2. Casos de error — deben fallar limpio (sin crashear)

Casos exigidos por el documento (sección 5.2 — Manejo de Casos de Error):

- Newton x^3 , $x_0=0$ $\square$ FAILED por derivada = 0 (división por cero).
- Newton $\log(x)$, $x_0=-1$ $\square$ FAILED por dominio matemático.
- Newton $x^{(1/2)}$, $x_0=-4$ $\square$ FAILED por dominio matemático.
- Secante con valores que produzcan denominador $\square 0$ $\square$ FAILED por división por cero.
- Cualquier método con `maxIter` muy bajo (ej. 2) en una función de convergencia lenta $\square$ DONE con "convergió: No" y mostrando las iteraciones parciales.

3. Validación del frontend

Estos casos NO deben crear el job — debe aparecer un error en rojo debajo del campo:

- Función vacía.
- Función con basura: hola, jeje, /, ,
- Texto en $x_0/x_1/x_2$: hola, abc
- Tolerancia inválida: texto, 0, o valor negativo.
- Secante/Müller con $x_0 = x_1$ (valores iniciales iguales).

- $\text{maxIter} = 0$ o negativo.

4. Formatos numéricos

La tolerancia debe parsear correctamente estos tres formatos:

- 0.001
- 1e-6
- 10^{-6}

Además, los valores con coma decimal (locale Guatemala) deben funcionar igual: 0,5 equivale a 0.5.

5. Arquitectura asíncrona (vale 3 pts en la rúbrica)

- Crear un job y verificar que la respuesta es inmediata (no se cuelga esperando el cálculo).
- Confirmar que el estado pasa por PENDING $\square$ RUNNING $\square$ DONE en el polling cada 5 s.
- Crear 3-4 jobs seguidos rápidamente $\square$ todos deben procesarse sin perderse.

6. Reproducibilidad (vale 2 pts en la rúbrica)

- Que otro integrante clone el repositorio limpio y lo levante con `docker compose up --build` sin pasos manuales $\square$ debe funcionar de cero.

Nota: Según el documento, faltan los métodos **Gauss** y **Gauss-Jordan** y la página de **sistemas lineales** (/sistemas), pendientes para la Revisión 3 (23 de mayo). Por ahora las pruebas aplican únicamente a los métodos de raíces (Newton-Raphson, Secante y Müller).

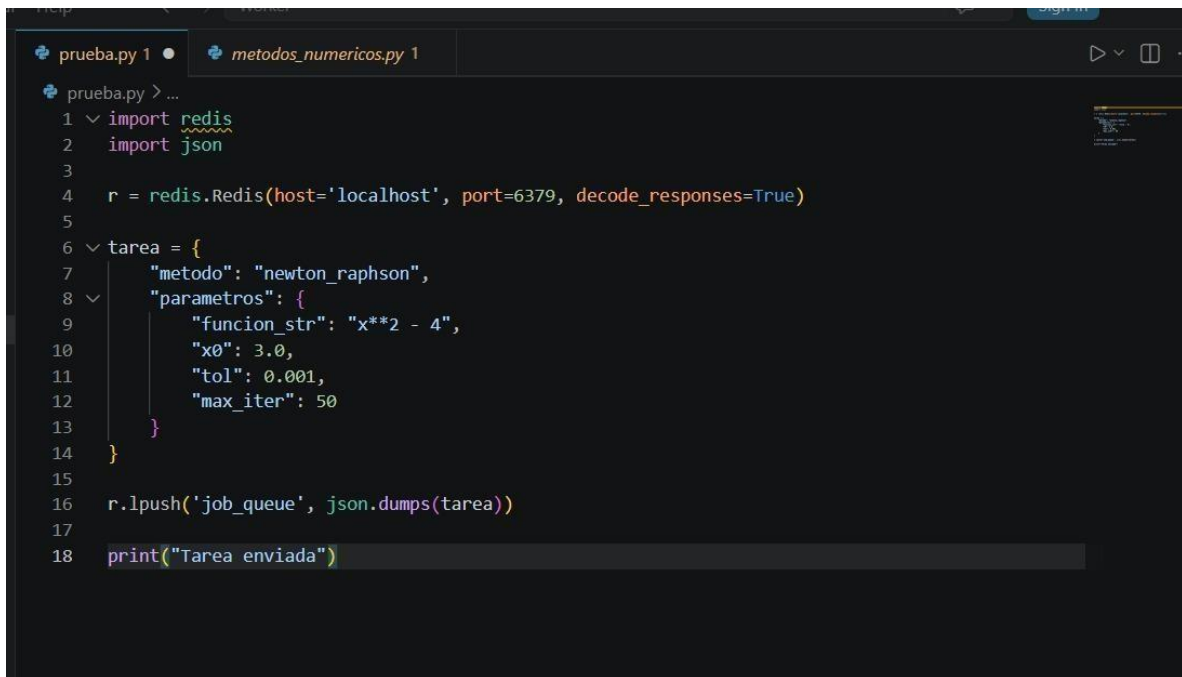
PRUEBA DE PROGRAMA PARA POSIBLES ERRORES

Encargados de pruebas:

Eduardo Leonel Monzón López 1790-24-
8668 César Gustavo Castillo García
1790-24-20456

PRUEBAS EN POWERSHELL:

Primera prueba:

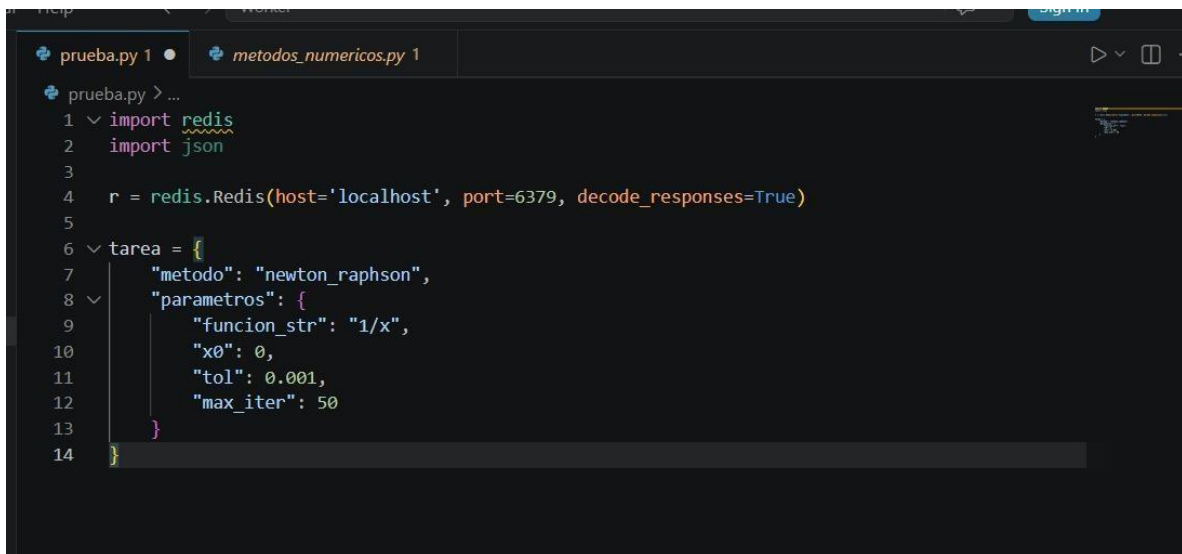


```
prueba.py > ...
1  import redis
2  import json
3
4  r = redis.Redis(host='localhost', port=6379, decode_responses=True)
5
6  tarea = {
7      "metodo": "newton_raphson",
8      "parametros": {
9          "funcion_str": "x**2 - 4",
10         "x0": 3.0,
11         "tol": 0.001,
12         "max_iter": 50
13     }
14 }
15
16 r.lpush('job_queue', json.dumps(tarea))
17
18 print("Tarea enviada")
```

Segunda prueba:

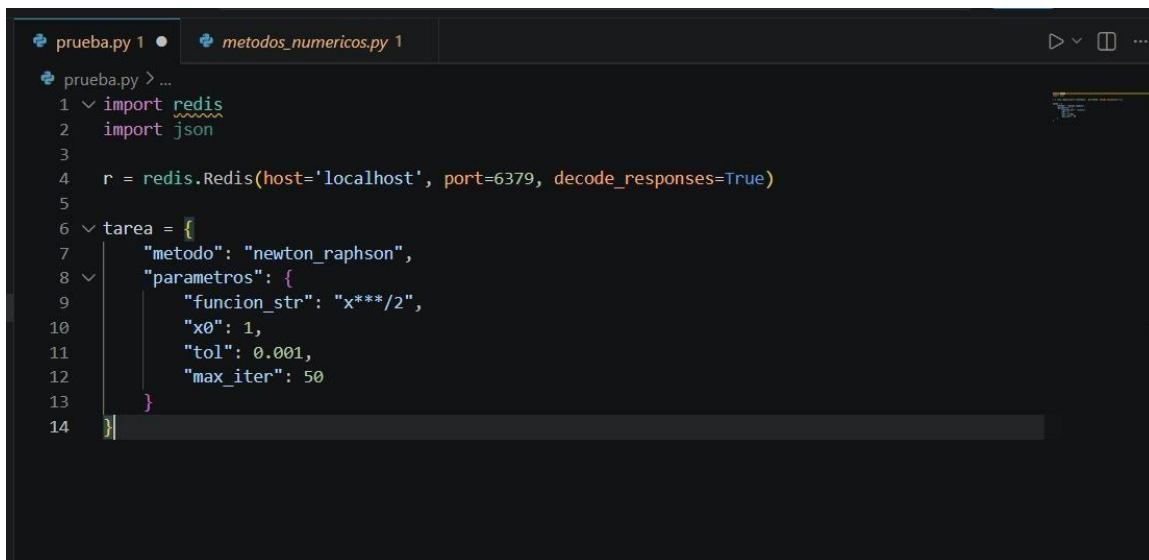
```
prueba.py 1 •  metodos_numericos.py 1
prueba.py > ...
1  import redis
2  import json
3
4  r = redis.Redis(host='localhost', port=6379, decode_responses=True)
5
6  tarea = {
7      "metodo": "metodo_magico",
8      "parametros": {
9          "x": 1
10     }
11 }
```

Tercera prueba:



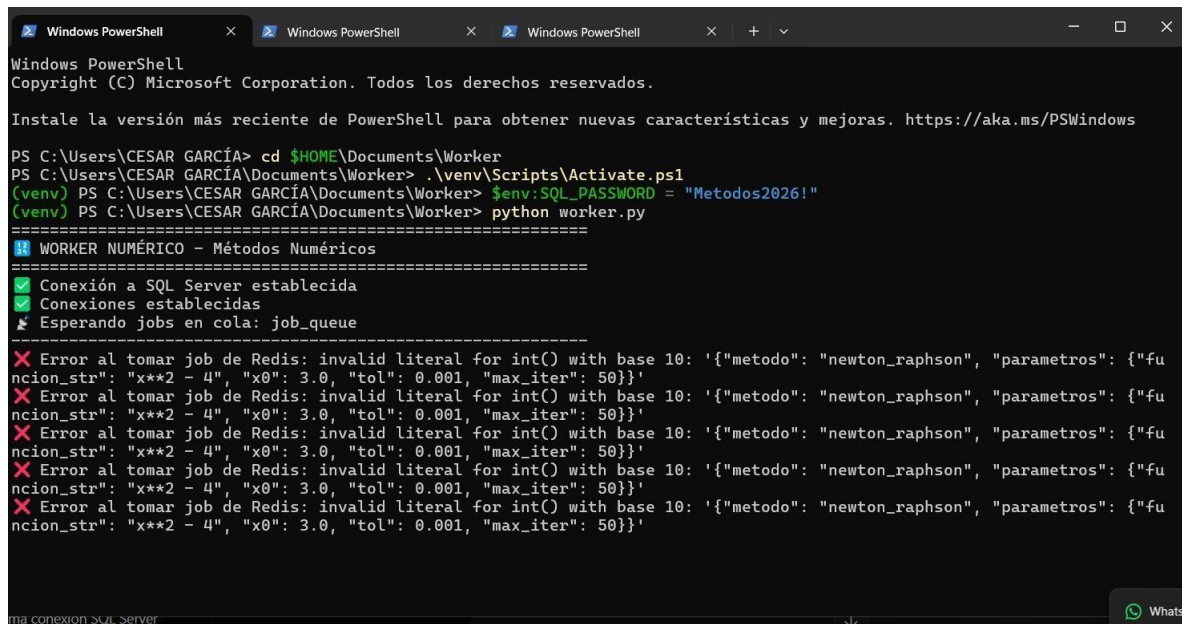
```
prueba.py > ...
1 import redis
2 import json
3
4 r = redis.Redis(host='localhost', port=6379, decode_responses=True)
5
6 tarea = {
7     "metodo": "newton_raphson",
8     "parametros": {
9         "funcion_str": "1/x",
10        "x0": 0,
11        "tol": 0.001,
12        "max_iter": 50
13    }
14 }
```

Cuarta prueba:



```
prueba.py > ...
1 import redis
2 import json
3
4 r = redis.Redis(host='localhost', port=6379, decode_responses=True)
5
6 tarea = {
7     "metodo": "newton_raphson",
8     "parametros": {
9         "funcion_str": "x**3/2",
10        "x0": 1,
11        "tol": 0.001,
12        "max_iter": 50
13    }
14 }
```

Captura General de Errores:



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\CESAR GARCÍA> cd $HOME\Documents\Worker
PS C:\Users\CESAR GARCÍA\Documents\Worker> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\CESAR GARCÍA\Documents\Worker> $env:SQL_PASSWORD = "Metodos2026!"
(venv) PS C:\Users\CESAR GARCÍA\Documents\Worker> python worker.py
=====
WORKER NUMÉRICO - Métodos Numéricos
=====
[X] Conexión a SQL Server establecida
[X] Conexiones establecidas
[X] Esperando jobs en cola: job_queue
=====
[X] Error al tomar job de Redis: invalid literal for int() with base 10: '{"metodo": "newton_raphson", "parametros": {"funcion_str": "x**2 - 4", "x0": 3.0, "tol": 0.001, "max_iter": 50}}'
[X] Error al tomar job de Redis: invalid literal for int() with base 10: '{"metodo": "newton_raphson", "parametros": {"funcion_str": "x**2 - 4", "x0": 3.0, "tol": 0.001, "max_iter": 50}}'
[X] Error al tomar job de Redis: invalid literal for int() with base 10: '{"metodo": "newton_raphson", "parametros": {"funcion_str": "x**2 - 4", "x0": 3.0, "tol": 0.001, "max_iter": 50}}'
[X] Error al tomar job de Redis: invalid literal for int() with base 10: '{"metodo": "newton_raphson", "parametros": {"funcion_str": "x**2 - 4", "x0": 3.0, "tol": 0.001, "max_iter": 50}}'
[X] Error al tomar job de Redis: invalid literal for int() with base 10: '{"metodo": "newton_raphson", "parametros": {"funcion_str": "x**2 - 4", "x0": 3.0, "tol": 0.001, "max_iter": 50}}'
```

Error estimado:

worker falla al leer tareas desde Redis porque intenta convertir a entero un mensaje JSON.

El dato recibido contiene parámetros del método numérico (newton_raphson) y debe deserializarse con `json.loads()` en lugar de usar `int()`

Durante las pruebas, el worker sí logró conectarse correctamente a SQL Server y Redis, pero al consumir un job desde `job_queue` falló con:

`invalid literal for int() with base 10`

El payload recibido era un JSON válido del método `newton_raphson`, por lo que el problema parece estar en el parsing/procesamiento interno del worker, donde se intenta convertir el contenido del job a entero en lugar de deserializar el JSON.

- Conexión SQL (OK)
- Conexión Redis (OK)
- Fallo en procesamiento del mensaje de cola (NO)
- Error de casteo/parsing en el worker (NO)

Error extra a considerar:

Complicaciones al abrir el programa, se modifico la contraseña de base de datos para que permitiera conexión (Falla de conexión a SQL SERVER).

Sistema de Métodos Numéricos

Evaluación funcional, matemática, visual y de manejo de errores

QA Tester	Eduardo Leonel Monzón López 1790-24-8668 César Gustavo Castillo García 1790- 24-20456
Proyecto	APLIMETODOS - Métodos numéricos
Entorno	Docker Compose, API .NET, Worker Python, SQL Server, Redis, Frontend React
Fecha del reporte	26/05/2026

Objetivo general

Documentar las pruebas realizadas al sistema, registrando resultados correctos, errores controlados, fallos matemáticos, problemas visuales y observaciones de calidad encontradas durante la ejecución de los métodos numéricos.

Reporte QA - Métodos Numéricos

1. Alcance de pruebas

Se evaluaron los siguientes módulos del sistema:

- Raíces: Newton-Raphson, Secante y Müller.
- Sistemas: Gauss, Gauss-Seidel y Gauss-Jordan.
- Comparador de métodos.
- Gráficas de convergencia.
- Tablas de iteraciones.
- Manejo de errores y entradas inválidas.

2. Resumen ejecutivo de hallazgos

Clasificación	Hallazgo
Correcto	Müller soporta raíces complejas como $1j$ y $4j$ sin colapsar.
Correcto	Secante detecta denominador cero cuando $x_0 = x_1$.
Correcto	Secante maneja overflow con estado FAILED y mensaje math range error.
Correcto	Gauss aplica pivoteo parcial cuando el pivote inicial es cero.
Correcto	Gauss-Jordan resuelve sistemas homogéneos y muestra pasos RREF.
Bug crítico	Secante puede marcar convergencia falsa y devolver raíz real en funciones sin raíz real.
Bug alto	Secante con $\tan(x)$ cerca de discontinuidad puede devolver $x = 0$ como convergencia.
Bug medio	Newton-Raphson no muestra correctamente $F(x_i)$, $F'(x_i)$ y E_a en la tabla.
Bug medio	Algunas tablas de Müller muestran columnas vacías aunque el cálculo se completa.
Mejora	Mensajes de error para funciones inválidas podrían ser más precisos.

3. Pruebas documentadas con resultados y errores del programa

Método	Entrada	Resultado esperado	Resultado real del programa	Conclusión QA
Secante	$\tan(x)$, $x_0=1.5$, $x_1=1.57$, $\text{tol}=1e-6$	Debería fallar o advertir discontinuidad cercana a $\pi/2$.	Convergencia: Sí. Raíz encontrada: 0.0000000000.	BUG ALTO: convergencia falsa hacia una raíz fuera de la zona inicial.
Secante	x^2+4 , $x_0=1$, $x_1=2$, $\text{tol}=1e-6$	No existe raíz real. Debe indicar no convergencia.	Convergencia: Sí. Raíz encontrada: -0.4398542477.	BUG CRÍTICO: devuelve raíz falsa, porque $f(x)$ no es cercano a cero.
Secante	$\exp(x^{100})$, $x_0=1$, $x_1=2$	Debe controlar overflow matemático.	FAILED: Error de evaluación en iteración 1: math range error.	Correcto: overflow manejado sin caída del sistema.
Secante	$\sin(1/x)$, $x_0=0.0001$, $x_1=0.0002$	Puede oscilar; debe mantenerse estable.	Convergencia: Sí. Raíz ≈ 0.3183098862 , 21 iteraciones.	Correcto: estabilidad ante función altamente oscilatoria.

Secante	hola, $x_0=2$, $x_1=1$	Debe rechazar función inválida.	Convergencia: No. Valor no finito en iteración 1. La función diverge o tiene una discontinuidad.	Correcto parcial: no colapsa, pero el mensaje debería decir función inválida.
Secante	$\sin(x)+\sin(x)+\sin(x)+\sin(x)+\sin(x)$, $x_0=1$, $x_1=1$	Debe detectar denominador cero.	Convergencia: No. Denominador cercano a cero en iteración 1. División por cero.	Correcto: validación matemática adecuada.
Müller	x^2+1 , $x_0=0$, $x_1=1$, $x_2=2$	Debe devolver raíz compleja i o -i.	Convergencia: Sí. Raíz encontrada: 1j.	Correcto: manejo de complejos.
Müller	x^2+16 , $x_0=1$, $x_1=2$, $x_2=3$	Debe devolver raíz compleja 4j o -4j.	Convergencia: Sí. Raíz encontrada: 4j.	Correcto: raíz compleja pura detectada.
Müller	x^2+1	Tabla debe mostrar X_0 , X_1 , X_2 y $F(X_2)$.	Tabla muestra guiones en varias columnas.	BUG MEDIO: frontend no renderiza todos los datos de iteración.
Gauss	Matriz $\begin{bmatrix} 0 & 2 & & 4 \\ 1 & 3 & & 5 \end{bmatrix}$	Debe aplicar pivoteo parcial y resolver.	Intercambio $F_1 \leftrightarrow F_2$. Solución $x_1=-1$, $x_2=2$.	Correcto: pivoteo parcial funcionando.
Gauss	Matriz $\begin{bmatrix} 0 & 1 & & -1 \\ 2 & 7 & & 1 \end{bmatrix}$	Debe aplicar pivoteo parcial.	Intercambio $F_1 \leftrightarrow F_2$. Solución $x_1=4$, $x_2=-1$.	Correcto: pivoteo y sustitución regresiva.

Reporte QA - Métodos Numéricos

Gauss-Jordan	Matriz [9 8 0], [7 5 0]	Sistema homogéneo con solución $x_1=0$, $x_2=0$.	Convergencia: Si. RREF mostrado paso a paso.	Correcto: sistema homogéneo resuelto.
Gauss-Seidel/Jordan	Campos con letras o vacíos	Debe validar entrada antes de calcular.	Error: float() argument must be a string or a real number, not NoneType.	BUG MEDIO: falta validación de celdas vacías o texto.
Comparador	x^{15} , $x_0=2$, $x_1=3$, $x_2=1$	Debe comparar y mostrar falla de cada método.	Newton sin datos de iteración/tiempo; Secante y Müller no convergen en 50 iteraciones.	Correcto parcial: comparador estable, pero visualización de Newton incompleta.
Newton-Raphson	Pruebas generales de raíz	Debe mostrar X_i , $F(X_i)$, $F'(X_i)$, X_{i+1} y E_a .	Solo mostró parcialmente $X_i/X_{i+1}/E_a$ según configuración.	BUG MEDIO: inconsistencia visual en tabla de iteraciones.

4. Errores principales reportados

ID	Severidad	Error	Descripción
E-01	Crítico	Secante devuelve raíz falsa en x^2+4	El sistema marcó convergencia aunque la función no tiene raíces reales. Resultado mostrado: $x^*=-0.4398542477$.
E-02	Alto	Secante con $\tan(x)$ cerca de $\pi/2$ devuelve $x=0$	El método debió advertir discontinuidad o no convergencia. En cambio marcó convergencia exitosa.
E-03	Medio	Müller muestra columnas vacías en tabla	En raíces complejas se muestran guiones para X_0 , X_1 , X_2 y $F(X_2)$, aunque el backend calcula el resultado.
E-04	Medio	Newton-Raphson no muestra correctamente columnas de tabla	No aparecen de forma consistente $F(X_i)$, $F'(X_i)$ y E_a .
E-05	Medio	Entradas vacías o texto en sistemas generan error de conversión	Mensaje: float() argument must be a string or a real number, not NoneType.
E-06	Bajo	Mensaje impreciso para función inválida	Para $f(x)=\text{hola}$ se reporta discontinuidad/divergencia en lugar de función inválida o variable no

			reconocida.
--	--	--	-------------

5. Validaciones correctas observadas

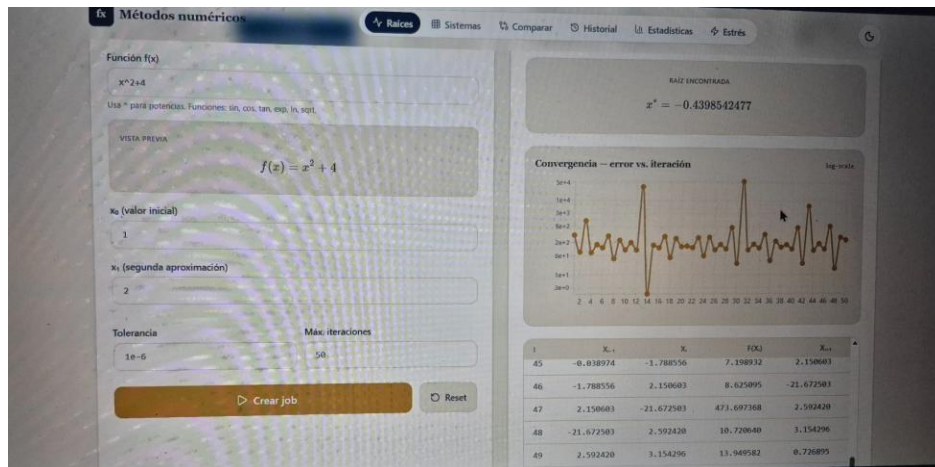
- El worker Python no se apagó durante las pruebas finales.
- Docker mantuvo los servicios activos con entradas inválidas y overflow.
- Secante detectó correctamente división por cero cuando $f(x_i) - f(x_{i-1}) = 0$.
- Müller manejó números complejos como $1j$ y $4j$.
- Gauss aplicó pivoteo parcial automáticamente con pivote inicial cero.
- Gauss-Jordan resolvió sistemas homogéneos y mostró normalización/eliminación.
- El módulo de comparación no colapsó aunque ningún método convergió en x^{15} .

6. Recomendaciones

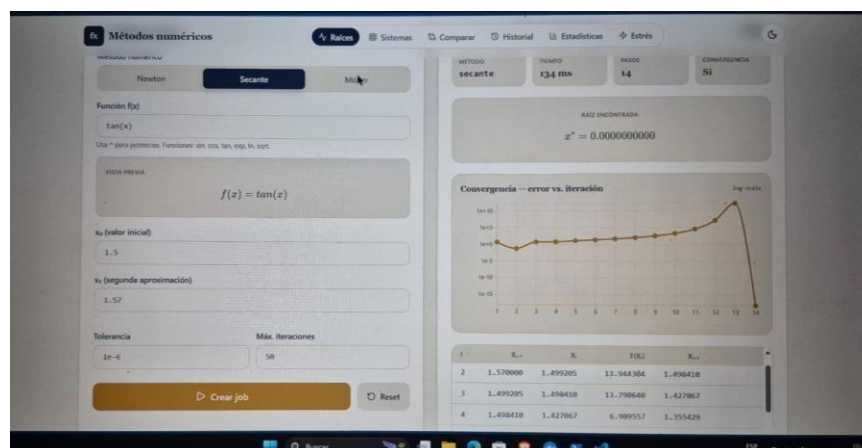
1. Agregar validación posterior de raíz: verificar que $|f(x^*)|$ sea cercano a cero antes de marcar convergencia.
2. Detectar discontinuidades y saltos extremos en Secante antes de aceptar convergencia.
3. Validar tolerancia negativa, cero o excesivamente grande desde frontend y backend.
4. Mejorar mensajes de error para funciones inválidas o variables no reconocidas.
5. Corregir mapeo de columnas de Newton-Raphson: $F(X_i)$, $F'(X_i)$, X_{i+1} y E_a .
6. Corregir tabla de Müller para mostrar X_0 , X_1 , X_2 y $F(X_2)$ cuando existen en los datos.
7. Validar matrices antes de enviar al backend para evitar NoneType o strings no numéricos.
8. En comparador, mostrar claramente por qué Newton falla o no entrega datos.

Reporte QA - Métodos Numéricos

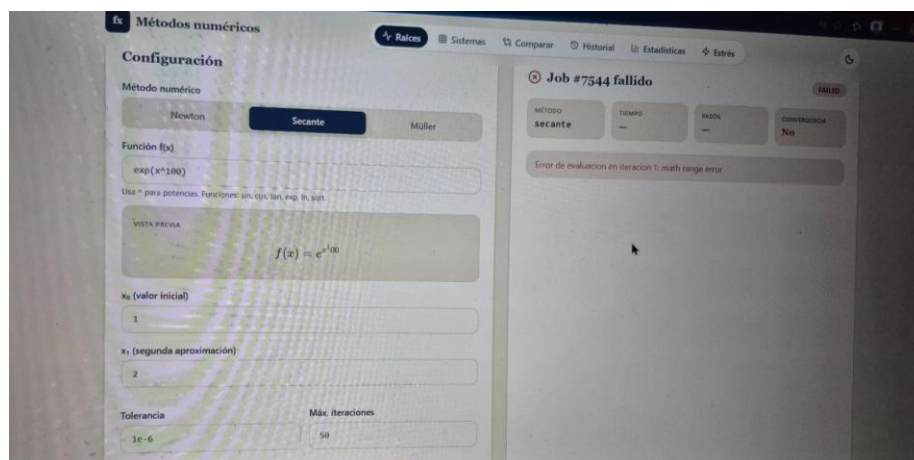
7. Evidencia visual



Evidencia E-01: Secante con x^2+4 devuelve una raíz falsa y marca convergencia.

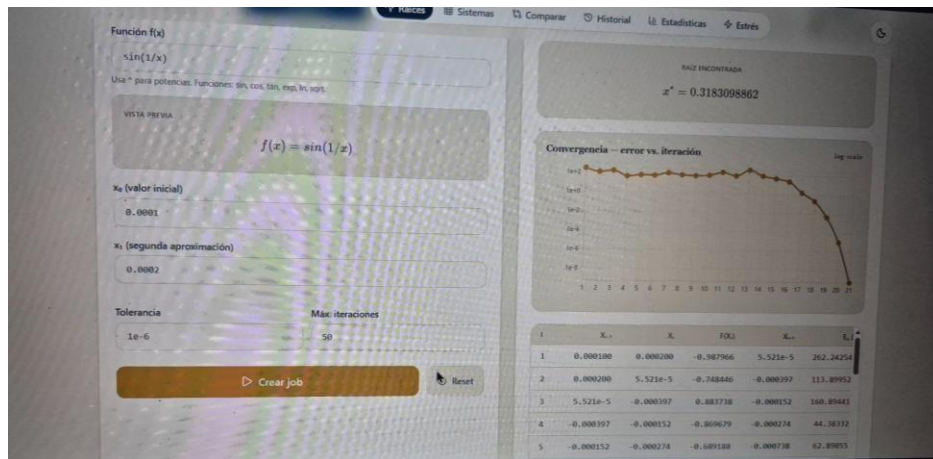


Evidencia E-02: Secante con $\tan(x)$ cerca de discontinuidad termina en $x=0$ con convergencia.

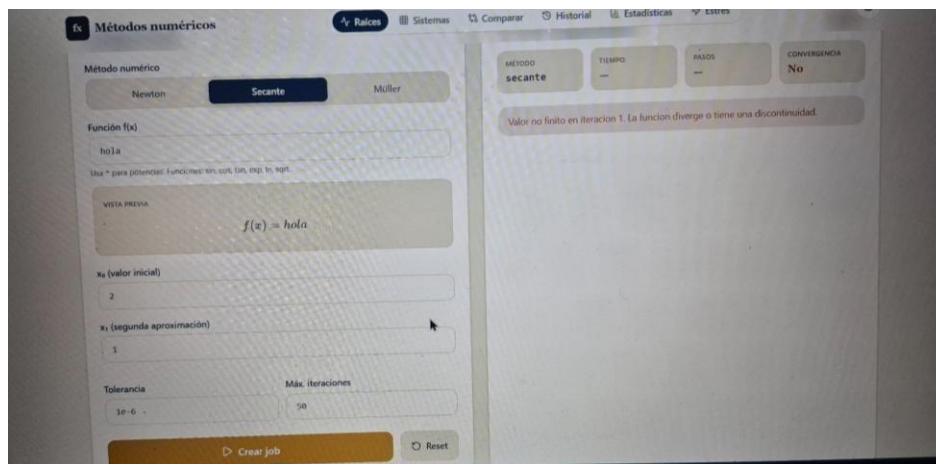


Evidencia: overflow controlado en Secante con $\exp(x^{100})$, estado FAILED.

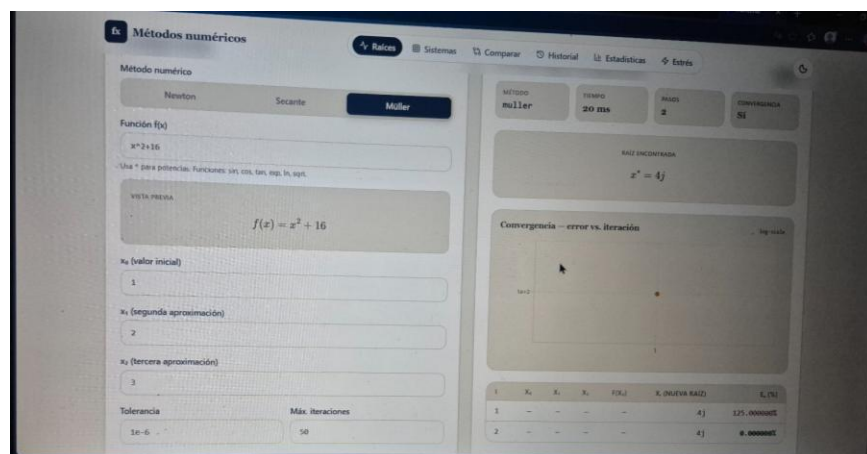
Reporte QA - Métodos Numéricos



Evidencia: Secante con $\sin(1/x)$ conserva estabilidad y converge tras múltiples iteraciones.

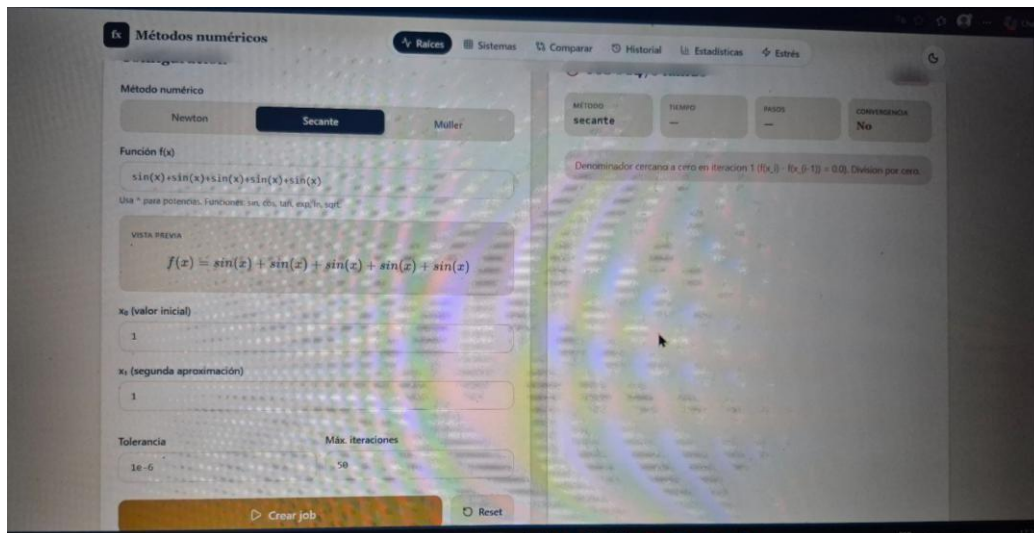


Evidencia: función inválida hola manejada sin caída, pero con mensaje mejorable.

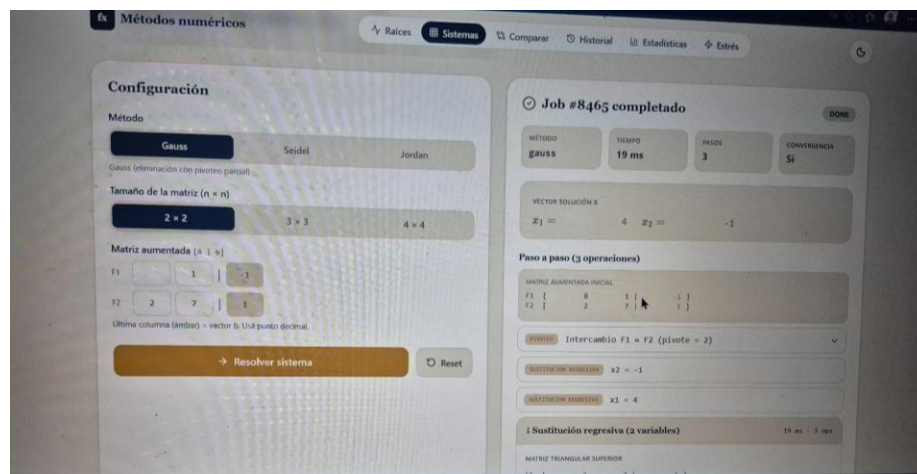


Evidencia: Müller resuelve x^2+16 y devuelve raíz compleja $4j$.

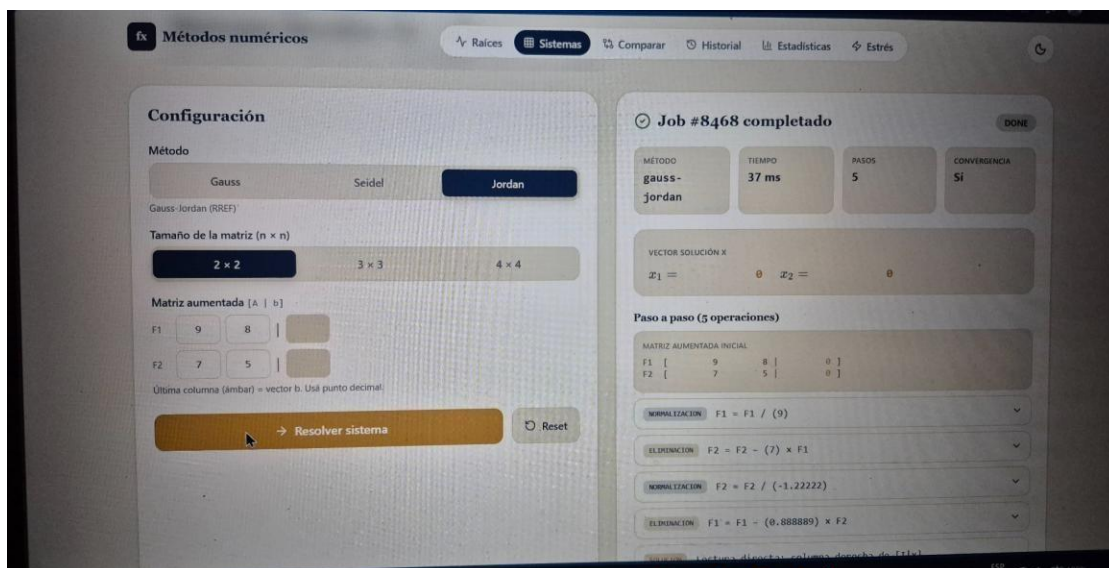
Reporte QA - Métodos Numéricos



Evidencia: Secante detecta denominador cero cuando $x_0=x_1$.

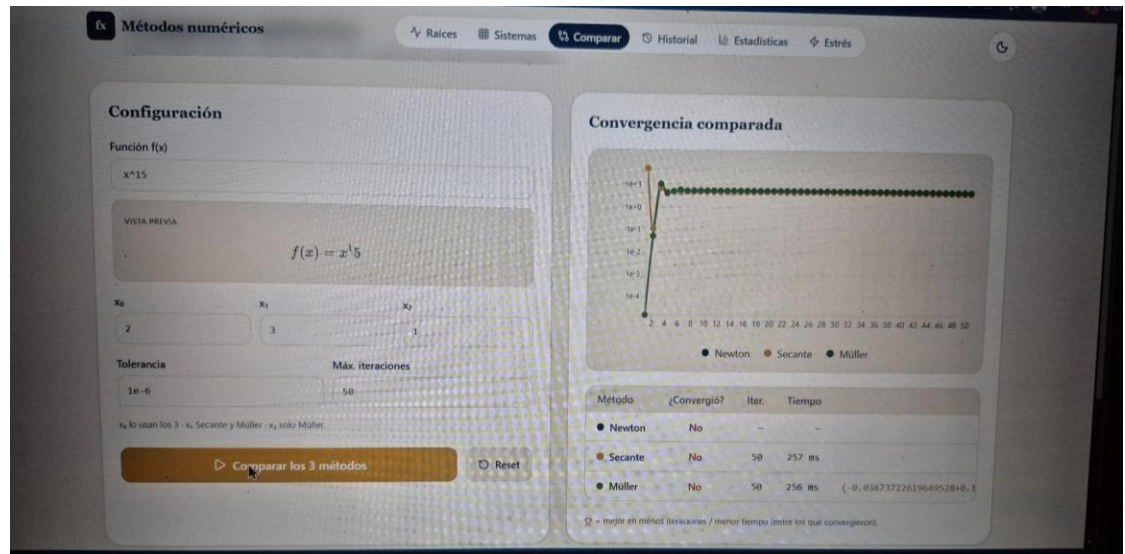


Evidencia: Gauss aplica pivoteo parcial y resuelve sistema 2x2.



Evidencia: Gauss-Jordan resuelve sistema homogéneo con RREF.

Reporte QA - Métodos Numéricos



Evidencia: Comparador con x^{15} muestra no convergencia y datos incompletos para Newton.

8. Conclusión final

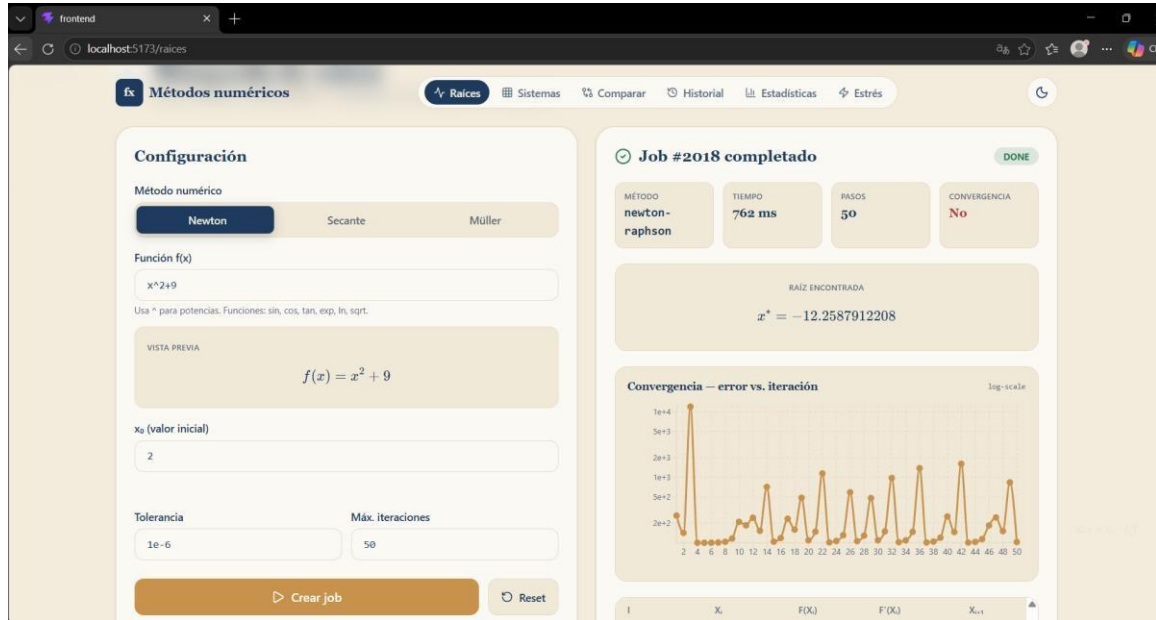
El sistema demostró estabilidad operativa general bajo Docker, con manejo correcto de varios errores matemáticos como overflow, división por cero y raíces complejas. Sin embargo, se identificaron errores importantes en la validación de convergencia, especialmente en el método de Secante, donde se llegaron a reportar raíces falsas como correctas. También existen problemas visuales en las tablas de Newton-Raphson y Müller. Se recomienda corregir estos puntos antes de aprobar completamente el proyecto.

REPORTE DE PRUEBAS Y ERRORES/BUGS

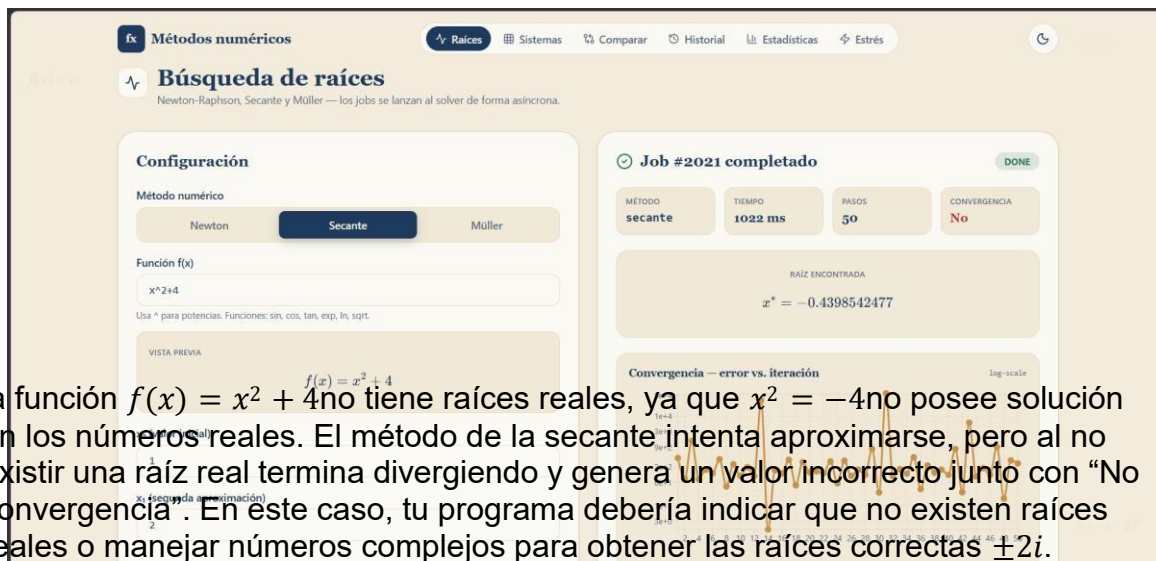
Proyecto: Métodos Numéricos

Responsable de pruebas:

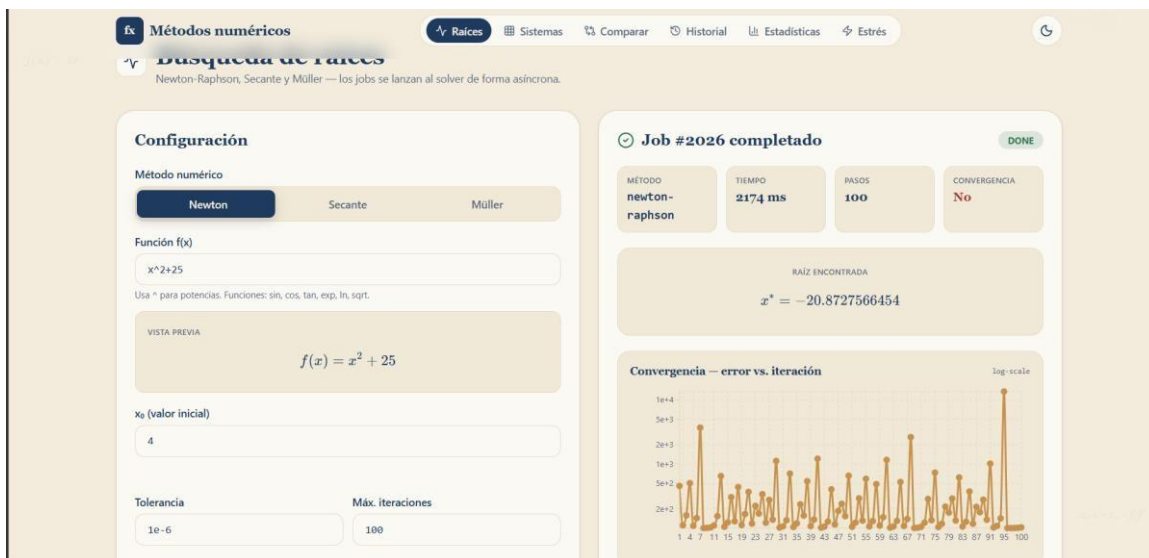
Eduardo Leonel Monzón López 1790-24-8668 Cesar Gustavo Castillo Garcia 1790-24-20456 **Fecha:** 27/05/2026



El resultado está mal porque la función $f(x) = x^2 + 9$ no tiene raíces reales, únicamente raíces complejas ($\pm 3i$). El método de Newton-Raphson comienza a divergir al acercarse a una derivada cercana a cero, por eso tu programa muestra valores muy grandes y "No convergencia". Lo correcto sería detectar que no existen soluciones reales o detener el proceso cuando la derivada sea demasiado pequeña. Aun así, el diseño y la interfaz de tu aplicación se ven muy profesionales y bien estructurados.



la función $f(x) = x^2 + 4$ no tiene raíces reales, ya que $x^2 = -4$ no posee solución en los números reales. El método de la secante intenta aproximarse, pero al no existir una raíz real termina divergiendo y genera un valor incorrecto junto con "No convergencia". En este caso, tu programa debería indicar que no existen raíces reales o manejar números complejos para obtener las raíces correctas $\pm 2i$.



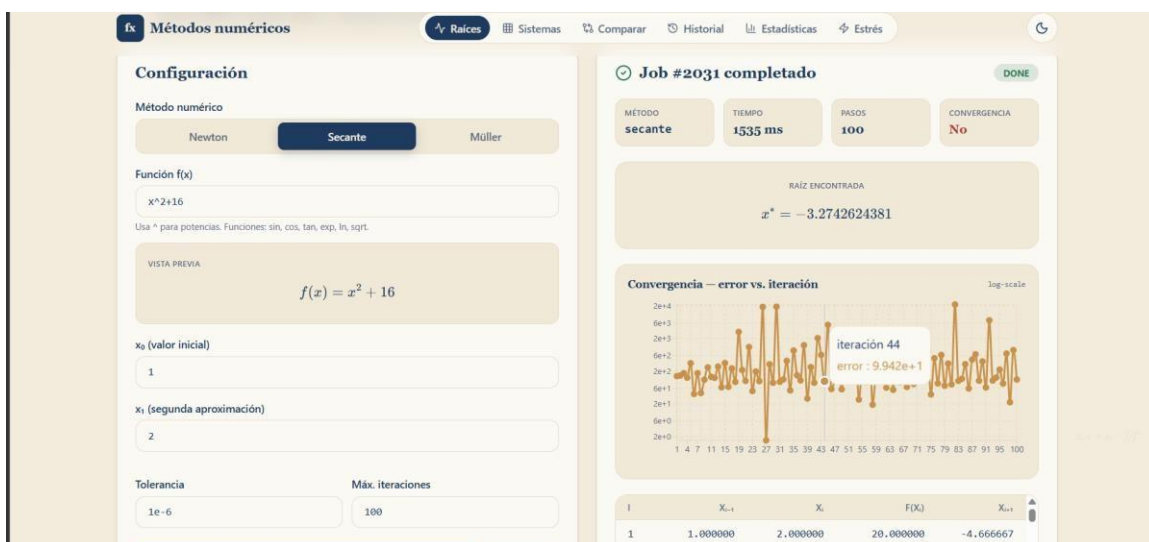
Está mal. Que el método “no converja” no significa que cualquier raíz negativa sea válida. La raíz solo es correcta si al reemplazarla en la función se cumple:

$$f(x) \approx 0$$

Por ejemplo, en $x^2 + 4$, si pruebas la “raíz” -0.4398 :

$$(-0.4398)^2 + 4 \approx 4.19$$

y eso está muy lejos de 0, así que no es una raíz real. Lo que ocurre es que los métodos iterativos están divergiendo y terminan mostrando números negativos aleatorios porque no encuentran solución real. Lo correcto es reportar “sin convergencia” o “no existen raíces reales”, no mostrar ese valor como raíz válida.



EN NUESTRA OPINION, DEBERIA DE MOSTRAR UN
MENSAJE DE QUE NO EXISTE RAIZ REAL Y NO DEBERIA
DE TIRAR RAIZ NEGATIVA